

Metodología de la Programación

Grado en Informatica

ÍNDICE

- 1. Programación Estructurada**
- 2. Ámbitos de visibilidad: globales, locales y de clase**
- 3. Funciones y métodos: Interfaces**
- 4. Parametrización. Parámetros por valor y por referencia**
- 5. Sobrecarga de métodos y operadores**
- 6. Constructores y destructores**
- 7. Atributos y métodos estáticos**
- 8. Consideraciones a tener en cuenta**

1. Programación Estructurada

- Un programa está compuesto de 1 a n **funciones**.
- **Función (subprograma)**: conjunto de acciones agrupadas bajo un nombre común.
- Las funciones:
 - Se escriben **una sola vez** y puede ser usadas **varias veces en distintas partes** del programa.
 - Hacen que un programa sea **más corto, fácil de entender y corregir y disminuye los errores**.
- Ventajas de uso de funciones:
 - **Modularización**.
 - **Ahorro** de memoria **y tiempo** de desarrollo.
 - **Independencia** de los datos y **ocultamiento** de la información.

Ejemplo:

```
// declaracion o prototipo  
double potencia(int base, int exponente);
```

Una función es conveniente **declararla**, para que el compilador sepa que existe y pueda chequear el nº y tipo de los argumentos y del valor de retorno.

```
...
```

En la **definición** se indica lo que hace la función, que tipo de valor retorna y cuántos y de que tipo son sus argumentos.

```
// definicion  
double potencia(int base, int exponente) {  
    double resultado;  
    resultado = ... ;  
    return resultado;  
}
```

La **llamada** se realiza incluyendo su nombre, y sus argumentos (datos pasados a la función).

```
...  
valor = potencia(2,3); // llamada  
valor = potencia(7,2); // llamada
```

2. Ámbitos globales, locales y de clase

Variable local

- **Es declarada dentro de un bloque o función.**
- Sólo puede ser usada dentro del bloque en el que está. No puede usarse fuera.
- Sólo existe durante la ejecución del bloque en el que está: se crea al entrar en el bloque, se destruye al salir.

Variable global

- **Es declarada fuera de cualquier bloque o función**, al inicio del programa
- Puede ser usada en cualquier parte del programa, y en cualquier función (no hay que pasarla como parámetro).
- Existe durante toda la ejecución del programa.

Una variable local y global pueden tener el mismo nombre (la variable local oculta a la global)

Variable miembro (atributo) de clase

Variable miembro privada

- **Es declarada en la parte private: de la clase.**
- No es visible fuera de la clase.
- Sólo la puede usar las funciones miembros (métodos) de la clase.

Variable miembro pública

- **Es declarada en la parte public: de la clase.**
- Es visible fuera de la clase.
- La puede usar los métodos de la clase y los objetos de dicha clase que haya en el programa.

Una variable miembro puede tener el mismo nombre que una variable local y una global (la variable local oculta a la de clase y a la global, la de clase oculta a la global)

Mediante el **operador de resolución de ámbito ::** podemos acceder a la que nos interese

::vble accede a la global

clase::vble accede a la de clase

Ejemplo:

```
#include <iostream>
using namespace std;
void ver();

class clase {
    int a;
public:
    int aget() { return a; }
    void aset(int i) { a = i; }
    void ver();
};

int a; clase x; // global

void clase::ver(){
    a=2; //de clase
    int a=0; //local
    a++; //local
    clase::a++; //de clase
    ::a=3; //global
    cout << "A " << ::a;
    cout << " X.a " << clase::a;
    cout << " a " << a << endl;
}
```

```
void ver(){
    a=0; //global
    x.aset(4); //global
    int a=5; //local
    clase x; //local
    a++; x.aset(2); //local
    ::x.aset(::x.aget()+1); ::a++; //global
    cout << "A " << ::a << " X.a " << ::x.aget();
    cout << " a " << a << " x.a " << x.aget() << endl;
}

int main(){
    a=2; x.aset(0); //global
    cout << "A " << a << " X.a " << x.aget() << endl;
    ver();
    cout << "A " << a << " X.a " << x.aget() << endl;
    x.ver();
    system("Pause"); return 0;
}
```

Salida del Programa:

```
A 2 X.a 0
A 1 X.a 5 a 6 x.a 2
A 1 X.a 5
A 3 X.a 3 a 1
```

3. Funciones y Métodos: Interfaces

3.1 Funciones genéricas

```
void funcion(int x, int y, char c) // bien
void funcion(int x, y, char c)    // mal
```

■ Sintaxis:

```
tipo nombre([tipo v1, ..., tipo vN]) {
    [declaraciones;]
    sentencias;
    [return valor_devuelto;]
}

if (cubo(x) > cuadrado(y))
    x = max(y, z) - raiz(9) + 7;
```

■ Las funciones:

- ☐ Pueden ser usadas en una expresión o asignación (excepto void).
- ☐ Existen durante todo el programa (tiempo de vida global).
- ☐ Pueden ser utilizadas en todo el programa (alcance global).
- ☐ Pueden ser recursivas y/o pueden llamar a otra función.

3.2 Métodos: funciones miembros de una clase

■ Sintaxis:

```
tipo nombre_clase::nombre_metodo([tipo v1, ..., tipo vN]) {
    [declaraciones;]
    sentencias;
    [return valor_devuelto;]
}

class complejo {
    int real, imag;
public:
    void ini(int r, int i);
    void mostrar();
};
```

■ Los métodos (funciones miembros):

- ☐ Tienen acceso directo (no hay que pasarlo como parámetro) a los **atributos privados** del objeto que lo invoca y a los **atributos privados de los objetos de su clase pasados como argumentos**.
- ☐ Pueden ser **públicos** o **privados**:
 - ☐ Los métodos **privados** sólo son accesibles por los métodos de la clase
 - ☐ Los métodos **públicos** son accesibles por los métodos de la clase y por los objetos de dicha clase definidos en el programa.

3.2.1 Métodos constantes (funciones miembros constantes)

■ Indican al compilador que el método no puede modificar el objeto que lo invoca

■ Sintaxis:

```
tipo nombre_clase::nombre_metodo([tipo v1, ..., tipo vN]) const {
    [declaraciones;]
    sentencias; //si alguna modifica algún atributo -> error
    [return valor_devuelto;]
}
```

- Un **método const** no puede modificar los atributos de su clase y **sólo puede llamar a otros métodos const de su clase** (aunque si puede llamar a **métodos no const** de otras clases)
- Los **objetos constantes (objetos const)** sólo pueden invocar los **métodos const de su clase** (al ser objetos constantes el compilador impide que puedan ser modificados y por tanto sólo permite que invoque los métodos **const** de su propia clase)
- Los **objetos y variables constantes sólo se pueden inicializar, no modificar**
`const Punto p(0,0);` //un punto de origen no debe cambiar nunca

Ejemplo:

Punto
- int x - int y
+ Punto(int a, int b) + operator char* () const + int getx () const + int gety () const + void setx (int n) + void sety (int n) - int min() const + void en(Punto &p) const + void clona(Punto p) + void cambia(Otra &o) const

Otra
- int x
+ void ini(int x=0) + int getx () const

Salida:

(1,2):(5,4):(2,3)
(5,4):(5,4):(4,3)
(4,3):(5,4):(5,4)

void Punto::en(Punto &p) const

- No modifica ningún atributo del objeto que lo invoca
- Modifica los atributos del objeto p pasado como parámetro

void Punto::cambia(Otra &o) const

- invoca a Punto::min() que es un método const de su clase
- invoca a Otra::ini() que es un método no const de otra clase
- No modifica ningún atributo del objeto que lo invoca

El objeto constante p2 solo puede invocar los métodos **const** de su clase

```
#include <cstdlib>
#include <iostream>

using namespace std;

class Otra {
    int x;
public:
    void ini(int x=0) { this->x=x; }
    int getx() const { return x; }
};

class Punto {
    int x, y;
    int min() const { return x<y?x:y; }
public:
    Punto(int a, int b) { x = a; y = b; }

    /*Los métodos de cambio de las coordenadas no
    pueden ser invocados por objetos constantes */
    void setx(int n) { x = n; } //const nunca
    void sety(int n) { y = n; } //const nunca
    void en(Punto &p) const { p.x=x; p.y=y; }
    void clona(Punto p) { x=p.x; y=p.y; } //const nunca
    void cambia(Otra &o) const { int n=min(); o.ini(n); }

    /*Los métodos de lectura de coordenadas sí
    pueden ser invocados por objetos constantes */
    int getx() const { return x; }
    int gety() const;

    /*La conversión de tipo si puede ser constante */
    operator char*() const {
        char salida[30];
        sprintf(salida, "(%i,%i)",x,y);
        return strdup( salida );
    }
};

int Punto::gety() const { return y; }

int main(int argc, char *argv[]) {
    Punto p1(1,2),p3(2,3);
    const Punto p2(5,4);
    Otra ot;
    cout << p1 << ":" << p2 << ":" << p3 << "\n";
    p3.setx(4);
    //p2.setx(100); //ERROR: el método no es constante
    p1.clona(p2);
    //p2.clona(p3); //ERROR: el método no es constante
    cout << p1 << ":" << p2 << ":" << p3 << "\n";
    p2.en(p3);
    p2.cambia(ot);
    p1.sety(p3.gety()-1);
    p1.setx(ot.getx());
    cout << p1 << ":" << p2 << ":" << p3 << "\n";
    //p1.min(); //ERROR min es privado
    system("PAUSE");
    return EXIT_SUCCESS;
}
```

3.2.1 Métodos constantes (sobrecarga)

- Un método puede sobrecargarse con una versión `const` y otra no `const`
- En tiempo de ejecución el compilador ejecutará una versión u otra dependiendo de si el objeto que invoca el método es un objeto constante o no

Punto

- int x
- int y

+ Punto(int a, int b)
+ operator char* () **const**
+ int getx () **const**
+ int gety () **const**
+ void setx (int n)
+ void sety (int n)
- int min() **const**
+ void clona(Punto &p) **const**
+ void clona(Punto p)
+ int proce() **const**
+ int proce()
+ void proce(int n) **const**

Sobrecarga método clona()

▪ void clona(Punto &p) **const**
▪ void clona(Punto p)

Sobrecarga método proce()

▪ int proce() **const**
▪ int proce()
▪ void proce(int n) **const**

Salida:

(1,2):(5,4):(2,3)
(5,4):(5,4):(5,4)
1,0
10
15

```
#include <cstdlib>
#include <iostream>

using namespace std;

class Punto {
    int x, y;
    int min() const { return x<y?x:y; }
public:
    Punto(int a, int b) { x = a; y = b; }

    void setx(int n) { x = n; } //const nunca
    void sety(int n) { y = n; } //const nunca

    { void clona(Punto &p) const { p.x=x; p.y=y; }
      void clona(Punto p) { x=p.x; y=p.y; }
    }

    { int proce() const;
      int proce();
      void proce(int n) const;
      //void fun() const;

      int getx() const { return x; }
      int gety() const { return y; }

      operator char*() const {
          char salida[30];
          sprintf(salida, "(%i,%i)",x,y);
          return strdup( salida );
      }
};

int Punto::proce() const {
    return 0;
}

int Punto::proce() {
    return 1;
}

void Punto::proce(int n) const {
    cout << n << endl;
}

//void Punto::fun() { //no es fun() const!!! }

int main(int argc, char *argv[]) {
    Punto p1(1,2),p3(2,3);
    const Punto p2(5,4);
    cout << p1 << ":" << p2 << ":" << p3 << "\n";
    p1.setx(5);
    //p2.setx(5); //ERROR
    p1.clona(p2);
    p2.clona(p3);
    cout << p1 << ":" << p2 << ":" << p3 << "\n";
    cout << p1.proce() << "," << p2.proce() << "\n";
    p1.proce(10);
    p2.proce(15);
    //p1.min(); //ERROR min es privado
    system("PAUSE");
    return EXIT_SUCCESS;
}
```

3.3 Especificador inline para funciones y métodos

- Indica al compilador que **sustituya la llamada a una función por su código**
 - La ejecución es más rápida, al no tener que transferir el control.
 - El programa ocupa más espacio al duplicar el código en cada llamada:

Existen 2 formas de definir las

- Poniendo la palabra **inline** en la definición (no declaración) del método o función

```
void ver(int a); // declaración
...
inline void ver(int a) {
    . . . // definición
}
```

`x = cuadrado(x) + doble(3);`
es sustituido por:
`x = x*x + 2*3;`

- Introduciendo el código de la función o método en la declaración

```
int doble(int x) { int d=2*x; return d; }
int cuadrado(int a) { return a*a; }
```

Ejemplo:

```
#include <iostream>
using namespace std;
```

```
int doble(int n); // prototipo funcion (declaracion)
int factorial(int n);
```

```
class complejo {
    int real, imag; // parte privada
```

```
public:
```

```
void ini(int re, int im) {real=re; imag=im;} // inline
```

```
void clon(complejo c) { real=c.real; imag=c.imag; } //inline
```

```
void suma(int re, int im);
```

```
void ver() const;
```

```
};
```

```
inline void complejo::ver() const {
    cout << real << "+" << imag << "i";
}
```

```
void complejo::suma(int re, int im) {
    real += re; imag = imag + im;
}
```

```
inline int doble(int n) { return 2*n; }
```

```
int factorial(int n) {
    int fact=1;
    for(int i=1; i <= n; i++) fact *= i;
    return(fact);
}
```

Salida del Programa:

x: 2+3i y: 1+2i z: 2+3i

c: 2+3i

x: 6+5i y: 12+122i

```
int main() {
    complejo x,y,z;
    z.ini(2,3); y.ini(1,2); x.clon(z);
    //x es 2+3i, y es 1+2i, z es 2+3i
    const complejo c=z; //ERROR si no inicializo
    int n = doble(6)-1; // n = 11
    cout << "\nx: "; x.ver();
    cout << " y: "; y.ver();
    cout << " z: "; z.ver();
    cout << "\nc: "; c.ver();
    x.suma(4,2);
    //c.clon(y); //ERROR c no puede cambiar
    y.suma(n,factorial(5));
    cout << "\nx: "; x.ver();
    cout << " y: "; y.ver();
    system("Pause");
    return 0;
}
```

Nota:

- Hay 2 funciones genéricas **doble** y **factorial**: **doble()** está definida como **inline**.
- Hay una clase **complejo** con varios métodos: **ini()**, **clon()** y **ver()** son **inline**.
- El método **clon()** accede directamente a los **atributos privados** de su argumento implícito (el objeto que invoca la llamada) y a los **atributos privados del objeto pasado como argumento**, al ser de su misma clase.
- El método **ver()** es constante y es el único método que puede invocar el **objeto const c**
- El objeto **c** es **const**: sólo puede ser inicializado y no puede invocar ningún método que modifique el objeto

3.4 Funciones amigas (friend)

- Una función amiga (*friend*) de una clase es una **función** o **método** que no **es miembro de la clase**, pero **tiene permiso para acceder a los elementos privados** de los objetos de dicha clase pasado como parámetro.
- Una función amiga de una clase puede ser un **método de otra clase** o puede ser una **función genérica** que no pertenece a ninguna clase.
- Una **función** o **método** puede ser amiga de cuantas clases se quiera.
- Para declararla amiga, es necesario incluir su prototipo, precedido por la palabra **friend**, en la clase para la que va a ser amiga (es indiferente ponerla en la parte pública o privada: las **funciones amigas** son siempre accesibles y la accesibilidad de los **métodos amigos** la determina la zona donde está definida en la clase a la que pertenece)
- Las funciones amigas violan el principio de encapsulamiento de la programación orientada a objetos, pero es la clase la que decide quiénes son sus amigos.

Punto
- int x - int y
+ Punto(int x=0, int y=0) + void pinta() const
friend Punto suma (Punto p1, const Punto &p2) friend void General::inc(Punto p, int x)

General
- int n
+ void inc(Punto p, int x) + void pinta() const

Salida:
(30, 15) (32)

Punto suma(Punto, const Punto &p2)
p2 referencia constante :
- No hace una copia - Al ser constante el compilador prohíbe que se modifique

```
#include <iostream>
using namespace std;

class Punto; //declaracion anticipada para que
class General {
    int n;
public:
    void inc(Punto p, int x); //no de error aqui
    void pinta() const { cout <<"(" << n << ")\n"; }
};

class Punto {
    friend Punto suma(Punto p1, const Punto &p2);
    friend void General::inc(Punto p, int x);
private:
    int x, y;
public:
    Punto( int nx=0, int ny=0 ){ x=nx; y=ny; }
    void pinta() const {cout <<"(" << x << "," << y << ")\n"; }
};
//inc hay que definirlo aquí tras definir Punto
void General::inc(Punto p, int x) {
    n=p.x+x; //Acceso a atributos privados de p!!
};

Punto suma(Punto p1, const Punto &p2 ) {
    Punto res;
    res.x = p1.x + p2.x; //Acceso atributos privados!!
    res.y = p1.y + p2.y;
    return res;
}

int main(int argc, char *argv[]) {
    Punto p1(10,5), p2(20,10), p3 = suma(p1, p2);
    General g;
    p3.pinta();
    g.inc(p3, 2);
    g.pinta();
    system("PAUSE");
    return EXIT_SUCCESS;
}
```

- Las funciones amigas no son estrictamente necesarias (todo lo que se puede conseguir con funciones amigas se puede conseguir con funciones no amigas que utilicen la interfaz pública de la clase). Su uso se justifica sólo por razones de eficiencia

Con funciones amigas	Sin funciones amigas
<pre>#include <iostream> using namespace std; class Punto; //declaracion anticipada para que class General { int n; public: void inc(Punto p, int x); //no de error void pinta() const { cout << n << endl; } }; class Punto { friend Punto suma(Punto p1, const Punto &p2); friend void General::inc(Punto p, int x); private: int x, y; public: Punto(int nx=0, int ny=0) {x=nx; y=ny;} void pinta() const { cout << "(" << x << ", " << y << ")\n"; } }; //inc hay que definirlo tras definir Punto void General::inc(Punto p, int x) { n=p.x+x; //Acceso a parte privada de p!! }; Punto suma(Punto p1, const Punto &p2) { Punto res; res.x = p1.x + p2.x; //acceso a res.y = p1.y + p2.y; //parte privada return res; } int main(int argc, char *argv[]) { Punto p1(10,5), p2(20,10); Punto p3 = suma(p1, p2); General g; p3.pinta(); g.inc(p3, 2); g.pinta(); system("PAUSE"); return EXIT_SUCCESS; }</pre>	<pre>#include <iostream> using namespace std; class Punto { private: int x, y; public: Punto(int nx=0, int ny=0) { x=nx; y=ny; } void pinta() const { cout << "(" << x << ", " << y << ")\n"; } int getx() const { return x; } int gety() const { return y; } }; class General { int n; public: void inc(Punto p, int x) { n=p.getx()+x; } void pinta() const { cout << n << endl; } }; Punto suma(Punto p1, const Punto &p2) { Punto res(p1.getx() + p2.getx(), p1.gety() + p2.gety()); return res; } int main(int argc, char *argv[]) { Punto p1(10,5), p2(20,10); Punto p3 = suma(p1, p2); General g; p3.pinta(); g.inc(p3, 2); g.pinta(); system("PAUSE"); return EXIT_SUCCESS; }</pre>

Nota:

- En la función genérica suma, p2 es una referencia **constante**: solo puede invocar métodos **constantes**, y por eso getx() y gety() son declarados **const**
 - tipo clase::método(parámetros) **const** método constante: el compilador impide que modifique los atributos del objeto que lo invoca
 - const tipo &p referencia constante: - el compilador impide que se modifique el objeto p
- no se pasa una copia del objeto sino el objeto

3.5 Clases amigas (friend)

- Una clase puede declararse como amiga de otra/s clase/s.
- Cuando una clase A se declara amiga de otra B, todos los métodos de la clase A pasan a ser amigos de la clase B.
- Es la forma más rápida de declarar todos los métodos de una clase como amigos de otra.

```
class Cualquiera {  
    . . .  
    friend class Amiga;  
};
```

Los métodos de la clase **Amiga** pueden acceder a la parte privada de los objetos de la clase **Cualquiera** pasados como parámetros, pero no al revés.

Punto

- int x
- int y

+ Punto(int x=0, int y=0)
+ void pinta() const
friend class Amiga

General

+ void inc(Punto p, int x)
+ void ponACero(Punto &p)

Salida:

(10, 5)
(0, 0)
0
-2

```
#include <iostream>  
using namespace std;  
  
class Punto {  
    friend class General; //clase amiga  
private:  
    int x, y;  
public:  
    Punto( int nx=0, int ny=0 ){ x=nx; y=ny; }  
    void pinta() const {cout <<"("<< x <<","<< y << ")\n";}  
};  
  
class General {  
    int n;  
public:  
    void ponACero(Punto &p) {  
        p.x = 0; //Acceso a parte privada de Punto  
        p.y = 0;  
        n = 0;  
    }  
    void inc(Punto p, int x) {  
        n = p.x + x; //Acceso a parte privada de Punto  
    }  
    void pinta() const { cout << n << endl; }  
};  
  
int main(int argc, char *argv[]) {  
    Punto p1(10,5);  
    General g;  
    p1.pinta();  
    g.ponACero(p1);  
    p1.pinta();  
    g.pinta();  
    g.inc(p1, -2);  
    g.pinta();  
  
    system("PAUSE");  
    return EXIT_SUCCESS;  
}
```

3.5 Clases amigas (friend) (caso de referencia cruzada)

- Si se quiere que dos clases tengan acceso mutuo a los miembros privados de los objetos de la otra clase pasados como parámetro, cada una debe declararse amiga de la otra.
- **Problema:** no podemos utilizar una clase antes de declararla.
- **Solución:** *declarar anticipadamente* (redirigir la declaración de) *la clase que definimos en segundo lugar*.
- **Cuando se redirige una clase, se pueden declarar variables de ese tipo y punteros.** Básicamente, podemos declarar los prototipos, pero como aún no se han declarado los atributos y métodos reales de la clase redirigida, no se pueden invocar. **La solución es sólo hacer las declaraciones primero y luego definir los métodos.**

Amiga1
- int privada
+ void modificaAmiga2 (Amiga2 &a2, int val) + void pinta() const friend class Amiga2 friend int suma(Amiga1 a1, Amiga2 a2)

Amiga2
- int privada
+ void modificaAmiga1 (Amiga1 &a1, int val) + void pinta() const friend class Amiga1 friend int suma(Amiga1 a1, Amiga2 a2)

Salida:
20 10 suma = 30

```
#include <iostream>
using namespace std;

class Amiga2; // declaración anticipada

class Amiga1 {
    friend class Amiga2;
    friend int suma(Amiga1 a1, Amiga2 a2);
private:
    int privada;
public:
    void modificaAmiga2(Amiga2 &a2, int val) const;
    /* Aquí no podemos definir el método porque aún
    el compilador no ha llegado a leer la definición de
    la clase Amiga2 y no sabe que atributos tiene... */
    void pinta() const { cout << privada << "\n"; }
};

class Amiga2 {
    friend class Amiga1;
    friend int suma(Amiga1 a1, Amiga2 a2);
private:
    int privada;
public:
    void modificaAmiga1(Amiga1 &a1, int val) const {
        /* Aquí si podemos porque ya hemos declarado Amiga1 */
        a1.privada = val;
    }
    void pinta() const { cout << privada << "\n"; }
};

int suma(Amiga1 a1, Amiga2 a2 ) {
    return (a1.privada+a2.privada);
}

/* Ahora sí que podemos definir el método porque ya
hemos declarado los atributos de la clase Amiga2 */

void Amiga1::modificaAmiga2(Amiga2 &a2, int val) const {
    a2.privada = val;
}

int main(int argc, char *argv[]) {
    Amiga1 aml;
    Amiga2 am2;
    aml.modificaAmiga2(am2,10);
    am2.modificaAmiga1(aml,20);
    aml.pinta(); am2.pinta();
    cout << "suma = " << suma(aml, am2) << "\n";
    system("PAUSE"); return EXIT_SUCCESS;
}
```

4. Parámetros por Valor y por Referencia

- Los **parámetros** son los valores que se le pasan a la función o método al ser llamada. Cada parámetro se comporta dentro de la función o método como una variable local.
- Los parámetros escritos en la llamada a la función se llaman **parámetros reales o argumentos**.
- Los que aparecen en la descripción de la función se llaman **parámetros formales**.
- Los parámetros formales son sustituidos por los reales (argumentos) en el orden de llamada. Al emparejarse éstos deben coincidir en número y en tipo.

4.1 Parámetros por Valor y por Referencia

Parámetros por valor o copia	Parámetros por referencia
■ Los cambios producidos dentro de la función no afectan a la variable usada como argumento en la llamada ■ En la llamada se pasa el valor (copia) de la variable usada como argumento. ■ El parámetro real puede ser una constante, variable o una expresión.	■ Los cambios producidos dentro de la función afectan a la variable con la que se realiza la llamada. ■ En la llamada se transfiere la propia variable. ■ El parámetro real sólo puede ser una variable, ya que se transfiere ésta. ■ Se indica poniendo el carácter & delante del nombre del parámetro formal.

Ejemplo:

```
#include <iostream>
using namespace std;

class obj; //declaracion anticipada
void permutar(obj &a, obj &b); //ref
void cambiamal(obj a, obj b); //valor

class obj {
    int n;
public:
    int get() const { return n; }
    void set(int i) { n = i; }
};

void permutar(obj &a, obj &b) {
    cout << "dir. memoria de a: " << &a << endl;
    cout << "dir. memoria de b: " << &b << endl;
    int aux=a.get();
    a.set(b.get());
    b.set(aux);
}

void cambiamal(obj a, obj b) {
    cout << "dir. memoria de a: " << a << endl;
    cout << "dir. memoria de b: " << b << endl;
    int aux=a.get();
    a.set(b.get());
    b.set(aux);
}
```

```
int main(){
    obj x, y;
    x.set(5); y.set(7);
    cout << "dir. memoria de x: " << &x << endl;
    cout << "dir. memoria de y: " << &y << endl;
    cambiamal(x, y);
    cout << "x: " << x.get()
        << " y: " << y.get() << endl;
    permutar(x, y);
    cout << "x: " << x.get()
        << " y: " << y.get() << endl;
    system("Pause"); return 0;
}
```

Salida del Programa:

```
dir. memoria de x: 0x61fe0c
dir. memoria de y: 0x61fe08
dir. memoria de a: 0x61fde0
dir. memoria de b: 0x61fde8
x: 5 y: 7
dir. memoria de a: 0x61fe0c
dir. memoria de b: 0x61fe08
x: 7 y: 5
```

Como podemos observar en el **main()** imprimimos la dirección de memoria del **objeto x** (0x61fe0c) y del **objeto y** (0x61fe08). En **cambiamal** pasamos los parámetros por copia: **a** (0x61fde0) es una copia de **x** (0x61fe0c) y **b** (0x61fde8) es una copia de **y** (0x61fe08). Por eso sus direcciones de memoria son distintas. En **permutar** pasamos los parámetros **por referencia**: la dirección de memoria de **a** y **x** es la misma (0x61fe0c) ya que no pasamos una copia, sino el original. Lo mismo ocurre con **b** e **y**.

4.2 Variables de tipo Referencia

- Es un alias o sinónimo de una variable (no ocupa memoria ya que no crea una nueva variable)
- Se declaran con el operador & y deben ser inicializadas a otra variable

```
int i=2;  
int& jref; //declaración no válida  
int& iref = i; //declaración válida  
iref = 6; //es lo mismo que poner i=6
```

La única variable creada es la variable i, iref es "otro nombre" con el que podemos hacer referencia a la variable i. Las direcciones de memoria de i (&i) y de iref (&iref) son las mismas (son lo mismo)

- Una función o método puede retornar una variable tipo referencia

```
int& maxref(int& a, int& b) {  
    if (a >= b) return a;  
    else return b; //con & devuelve b, no una copia de b  
} //sin & devuelve una copia de b, no b
```

- Al devolver una referencia, la llamada a la función o método puede estar a la izquierda del operador de asignación:

```
maxref(i, j) = 0; // asigna un 0 al parametro mayor
```

- Peligro al retornar referencias: el retorno puede estar indefinido si no programamos bien

```
int& malprogramado(int n) {  
    return n; //retorna n que al ser local es destruido al terminar  
}
```

4.3 Referencias constantes

Cuando la variable que queremos pasar como parámetro por valor a una función o método ocupa mucha memoria, se suele pasar por referencia por motivos de eficiencia, ya que al pasarlo por valor el compilador tiene que realizar una copia del parámetro y al pasarlo por referencia, mediante un puntero o un alias (referencia), no se pierde tiempo ni memoria en hacer una copia, sino que se pasa directamente el original.

Para evitar que accidentalmente se modifique el parámetro (al ser pasada por referencia, en vez de por valor) dentro de la función, es conveniente declarar el parámetro constante (**const**).

- El especificador **const** se puede utilizar tanto con variables (y referencias) como con punteros (no confundir punteros constantes con punteros a constantes).
- Una variable **const**, una referencia **const** o un puntero a una variable **const** no pueden cambiar el valor durante la ejecución del programa (sólo puede ser inicializado).
- Un objeto **const**, referencia **const** o puntero a objeto **const** sólo puede invocar métodos **const** y no puede cambiar durante la ejecución del programa (sólo puede ser inicializado)

Recuerda:

Si por motivos de eficiencia, en vez de pasar un objeto por valor lo pasamos por referencia (para evitar que el compilador tenga que realizar una copia del parámetro)

&&

por motivos de seguridad, para evitar que el parámetro pueda ser modificado (cuando se pasa por valor no importa ya que es una copia) la referencia la declaramos constante



debemos etiquetar como constantes aquellos métodos que lo sean, ya que si no, no podrán ser invocados por el parámetro por referencia constante

Diferencia entre Referencias constantes, Punteros constantes y Punteros a constantes

Las variables y las referencias pueden ser constantes.

Los punteros pueden ser constantes o apuntar a variables constantes (no es lo mismo).

- Las variables `const` y las referencias `const` no pueden cambiar su valor.
- Los punteros `const` pueden cambiar el valor de la variable a la que apuntan, lo que no pueden es apuntar a un sitio diferente.
- Los punteros a variables `const` no pueden cambiar el valor de la variable a la que apuntan, pero pueden apuntar a variables distintas a lo largo de la ejecución del programa.

Salida por pantalla:

```
6 7 7
2 5 5
7 5 7
4 8 4
-----
8 8 8
9 8 9
```

```
#include <iostream>
using namespace std;

int main() {
    int x, y;
    int *z;           //z es un puntero a un entero
    z=&x; *z=6;        //z puede apuntar a un sitio y modificar su contenido
    z=&y; *z=7;        //y luego apuntar a otro sitio y modificar su contenido
    cout << x << " " << y << " " << *z << endl;

    const int *a;     //a es un puntero a una constante entera
    a=&x;              //a puede apuntar a un sitio
    a=&y;              //y luego apuntar a otro sitio

    /*a=6;            //ERROR a no puede modificar el contenido al que apunta
    x=2; y=5;         // ya que considera ese contenido constante (lo sea o no)
    cout << x << " " << y << " " << *a << endl;

    int *const b=&x;   //b es un puntero constante a un entero. Al declararlo tiene
                        //que apuntar a algo y no puede apuntar a ningun otro sitio
    *b=6;              //el sitio al que apunta se puede
    *b=8;              //modificar cuantas veces quiera
    //b=&y;            //ERROR b no puede apuntar a otro sitio, el puntero es constante
    *b=7;
    cout << x << " " << y << " " << *b << endl;

    x=4; y=8;
    const int * const c=&x; //c es un puntero constante a una constante entera
    //c=&y;              //ERROR al declararlo tiene que apuntar a algo y no puede
                        //apuntar a ningun otro sitio
    /*c=6;            //ERROR tampoco va a poder modificar el contenido al que
                        //apunta ya que considera ese contenido constante

    cout << x << " " << y << " " << *c << endl;
    cout << "-----\n";

    //int &d;          //ERROR d es una referencia: hay que indicar a qué referencia
    int &d=x;          //d es una referencia a x (d es un alias de x)
    d=6;              //equivale a poner x=6
    d=y;              //equivale a poner x=y
    cout << x << " " << y << " " << d << endl;

    const int &e=x;    //e es una referencia a una constante (e es una alias de x)
    //e=8;            //ERROR e no puede modificar x porque considera que es constante
    x=9;              //pero x se puede seguir modificando
    cout << x << " " << y << " " << e << endl;
}
```

Ejemplo: Supongamos una clase que tiene una gran cantidad de datos...

uso habitual (menos eficiente)	uso de referencias constantes (más eficiente)
<pre>#include <iostream> #include <iostream> using namespace std; class General { private: int x[1000], n; public: General(int n, int ini, int inc); int getn() const { return n; } int getx(int i) const { return x[i]; } int setx(int i, int v) { x[i]=v; } void ver()const { for(int i=0;i<n;i++) cout << x[i] << " "; } bool compara(General g) const; }; bool General::compara(General g)const{ bool v = (n==g.n); for (int i=0; v && i<n; i++) v = x[i]==g.x[i]; return v; //g.n=0 no importa, es copia } General::General(int n, int ini, int inc) { this->n=n; for(int j=0,i=ini; j<n; i+=inc, j++) x[j]=i; } General suma(General p1, General p2) { int a=p1.getn(), b=p2.getn(); int n=a<b?a:b; General res(n, 0, 0); for(int i=0;i<n;i++) res.setx(i, p1.getx(i)+p2.getx(i)); return res; } int main(int argc, char *argv[]) { General a(5,-4,2), b(3,2,5), c=suma(a,b); c = suma(a, b); cout << "a: "; a.ver(); cout << endl; cout << "b: "; b.ver(); cout << endl; cout << "c: "; c.ver(); cout << endl; if (!a.compara(b)) cout << "a<>b\n"; a=b; cout << "a: "; a.ver(); cout << endl; cout << "b: "; b.ver(); cout << endl; if (a.compara(b)) cout << "a = b\n"; system("PAUSE"); return EXIT_SUCCESS; }</pre>	<pre>#include <iostream> #include <iostream> using namespace std; class General { private: int x[1000], n; public: General(int n, int ini, int inc); int getn() const { return n; } int getx(int i) const { return x[i]; } int setx(int i, int v) { x[i]=v; } void ver()const { for(int i=0; i<n; i++) cout << x[i] << " "; } bool compara(const General &g) const; }; bool General::compara(const General &g)const { bool v = (n==g.n); for (int i=0; v && i<n; i++) v = x[i]==g.x[i]; return v; //g.n=0 no permitido, es const } General::General(int n, int ini, int inc) { this->n=n; for(int j=0,i=ini; j<n; i+=inc, j++) x[j]=i; } General suma(const General &p1, const General &p2){ int a=p1.getn(), b=p2.getn(); int n=a<b?a:b; General res(n, 0, 0); for(int i=0;i<n;i++) res.setx(i, p1.getx(i)+p2.getx(i)); return res; } int main(int argc, char *argv[]) { General a(5,-4,2), b(3,2,5), c=suma(a,b); c = suma(a, b); cout << "a: "; a.ver(); cout << endl; cout << "b: "; b.ver(); cout << endl; cout << "c: "; c.ver(); cout << endl; if (!a.compara(b)) cout << "a<>b\n"; a=b; cout << "a: "; a.ver(); cout << endl; cout << "b: "; b.ver(); cout << endl; if (a.compara(b)) cout << "a = b\n"; system("PAUSE"); return EXIT_SUCCESS; }</pre>

Salida:

```
a: -4 -2 0 2 4
b: 2 7 12
c: -2 5 12
a <> b
a: 2 7 12
b: 2 7 12
a = b
```

- suma() al usar referencias constantes 'obliga' a etiquetar getn() y getx() como const
- suma() y compara() son más eficientes usando referencias constantes
- aunque no es obligatorio, ver() y compara() se etiquetan const ya que no modifica la clase
- en izq no importa (es copia), en dcha si importa (es original) y no está permitido al ser const

4.4 Valores de retorno constantes. Consideraciones

- Una referencia (o un puntero) a un atributo privado permite el acceso y/o modificación de éste desde fuera de la clase!!!!.
- Permite “ahorrar” métodos ya que la referencia podemos usarla para consultar y modificar.

Clase con métodos sin referencias devueltas	Clase equivalente con referencias devueltas
<pre>#include <iostream> using namespace std; class Punto { int x, y; public: Punto(int a, int b) { x=a; y=b; } int getx() const { return x; } int gety() const { return y; } void set(int a, int b) { x=a; y=b; } }; int main(int argc, char *argv[]) { Punto x(3,2); cout <<x.getx()<<" "<<x.gety()<<endl; x.set(5,7); cout <<x.getx()<<" "<<x.gety()<<endl; system("PAUSE"); return EXIT_SUCCESS; }</pre>	<pre>#include <iostream> using namespace std; class Punto { int x, y; public: Punto(int a, int b) { x=a; y=b; } int& refx() const { return x; } int& refy() const { return y; } //si devuelve referencia no puede ser const }; int main(int argc, char *argv[]) { Punto x(3,2); cout <<x.refx()<<" "<<x.refy()<<endl; x.refx()=5; x.refy()=7; cout <<x.refx()<<" "<<x.refy()<<endl; system("PAUSE"); return EXIT_SUCCESS; }</pre>

- Peligro: Supongamos que x debe ser siempre positivo (si es negativo ponerlo positivo)

Clase robusta	Clase no robusta (fallo de seguridad)
<pre>class Punto { int x, y; public: Punto(int a, int b) { set(a,b); } int getx() const { return x; } int gety() const { return y; } void set(int a, int b) { x=a; y=b; if (a<0) x=-a;} }; int main(int argc, char *argv[]) { Punto x(3,2); cout <<x.getx()<<" "<<x.gety()<<endl; x.set(-5,7); //no modifica al ser negativo cout <<x.getx()<<" "<<x.gety()<<endl; system("PAUSE"); return EXIT_SUCCESS; }</pre>	<pre>class Punto { int x, y; public: Punto(int a, int b) { x=a; y=b; if (a<0) x=-a;} int& refx() const { return x; } int& refy() const { return y; } void set(int a, int b) { x=a; y=b; if (a<0) x=-a;} }; int main(int argc, char *argv[]) { Punto x(3,2); cout <<x.refx()<<" "<<x.refy()<<endl; x.refx()=-5; x.refy()=7; //negativo!!! cout <<x.refx()<<" "<<x.refy()<<endl; system("PAUSE"); return EXIT_SUCCESS; }</pre>

- Solución: no retornar referencias o punteros desde un método cuando haya restricciones.

Si queremos devolver referencias o punteros **por motivos de eficiencia (para ahorramos hacer una copia si lo devuelto ocupa mucha memoria) o porque no hay más remedio (para devolver una cadena de caracteres hay que devolver un puntero a dicha cadena)** y queremos evitar que desde fuera de la clase se pueda modificar **debemos devolverlo como una referencia constante o como un puntero a una constante**

```
struct enorme {
    int T[1000];
    float f;
};

class General {
    char *c; //puntero a cadena caracteres
    enorme x; //ocupa mucha memoria
public:
    const char *getcad() const { return c; }
    const enorme& refx() const { return x; }
    enorme getx() const {return x;} //ineficiente
};
```

En método **refx()** es mas eficiente que **getx()** ya que evita devolver una copia de x, que ocupa mucha memoria

4.4 Valores de retorno constantes. Consideraciones

- Cuando un método devuelve una referencia de un atributo privado o un puntero de un atributo privado (bien por eficiencia, para evitar hacer una copia, o bien porque no hay mas remedio) debemos etiquetar esa referencia o puntero como **const** (referencia constante/puntero a constante) para evitar que desde fuera se pueda modificar el atributo.

problema	solución
<pre> #include <iostream> #include <cstring> using namespace std; class cadena { char *cad; int x[5], y; public: cadena(int ini, int incr, char cad[]); int *leer() const { return x; } int &leer0() const { return x[0]; } void ver() const; char *leercad() const { return cad; } int &refy() const { return y; } int *puny() const { return &y; } }; void cadena::ver() const { cout << cad << ":"; for(int i=0;i<5;i++) cout << x[i]; cout << " -> " << y << endl; } cadena::cadena(int ini, int incr, char cad[]){ y=0; for(int j=0,i=ini; j<5; j++,i+=incr) { x[j]=i; y+=x[j]; } this->cad = new char[strlen(cad)+1]; strcpy(this->cad, cad); } int main(int argc, char *argv[]) { char *c; int *pn; cadena x(1,1,"cadena x"); x.ver(); c=x.leercad(); strcpy(c,"CADENA X"); //x.cad="CADENA X" x.leer()[2]=0; //x.x[2]=0 x.refy()++; //x.y++ *(x.puny())+=2; //x.y+=2 x.ver(); int &n=x.refy(); n=4; //x.y=4 pn=x.puny(); (*pn)--; //x.y-- n=n*2; //x.y=x.y*2 x.leer0()=8; //x.x[0]=8 int *p=&(x.leer0()); p[1]=9; //x.x[1]=9 x.ver(); cout << x.leercad() << x.refy() << "\n"; system("PAUSE"); return EXIT_SUCCESS; } </pre>	<pre> #include <iostream> #include <cstring> using namespace std; class cadena { char *cad; int x[5], y; public: cadena(int ini, int incr, char cad[]); const int *leer() const { return x; } const int &leer0() const { return x[0]; } void ver() const; const char *leercad() const { return cad; } const int &refy() const { return y; } const int *puny() const { return &y; } }; int main(int argc, char *argv[]) { char *c; int *pn; cadena x(1,1,"cadena x"); x.ver(); c=x.leercad(); //ERROR no ok: uso indebido strcpy(c,"CADENA X"); //x.cad="CADENA X" x.leer()[2]=0; //x.x[2]=0 ERROR x.refy()++; //x.y++ ERROR no ok: uso indebido *(x.puny())+=2; //x.y+=2 ERROR x.ver(); int &n=x.refy(); n=4; //x.y=4 ERROR pn=x.puny(); (*pn)--; //x.y-- ERROR n++; //x.y++ ERROR x.leer0()=8; //x.x[0]=8 ERROR int *p=&(x.leer0()); p[1]=9; //ERROR x.ver(); cout << x.leercad() << x.refy() << "\n"; //ok system("PAUSE"); return EXIT_SUCCESS; } </pre> Con el puntero char *c puedo modificar el atributo privado char *cad!!! Con el puntero int *pn puedo modificar el atributo privado int y!!! Con la referencia int &n puedo modificar el atributo privado int y!!! Con los métodos leer(), leer0() puedo modificar la tabla privada int x[5]!!! Salida: <pre> cadena x:12345 -> 15 CADENA X:12045 -> 18 CADENA X:89045 -> 6 CADENA X6 </pre>

5. Sobrecarga de Métodos y Operadores

- C++ permite sobrecargar funciones, métodos y operadores, es decir, permite que existan varias funciones, métodos y operadores con el mismo nombre

- **Ventajas:**

- ☐ Ayuda a reducir la complejidad de un programa, ya que permite usar el mismo nombre y/o operador para operaciones relacionadas, (más intuitivo).
- ☐ Permite que funciones con el mismo nombre hagan cosas diferentes (polimorfismo)
- ☐ Permite utilizar los mismos operadores (y realizar las mismas operaciones) con los nuevos tipos de datos (clases) que creemos (siempre que redefinamos el operador)

5.1 Sobrecarga de funciones

- C++ permite definir varias funciones distintas con un mismo nombre, **siempre y cuando** difieran en el número y/o el tipo de sus argumentos

- El número y/o tipo de los argumentos de la llamada indicará la función a usar.

Si ninguna de las funciones se adapta a los parámetros indicados, se aplicarán las reglas implícitas de conversión de tipos.

Si las funciones sólo difieren en el tipo de datos que devuelven no se pueden sobrecargar:

```
int funcion(int x);
float funcion(int x);
int main() {
    . . . //ambigüedad
    funcion(2); //a cual llama?
}
```

Si sólo difieren en el tipo de sus argumentos, puede ser ambiguo, aunque la función no sea ambigua:

```
float dividir(float x);
double dividir(double x);
int main() {
    . . . //ambigüedad
    dividir(15); //15 double o float?
}
```

Ejemplos:

```
#include <iostream>
using namespace std;

int vabs(int n);
long vabs(long n);
double vabs(double n);

int main() {
    long b=-50000;
    cout << "\n|-6|: " << vabs(-6);
    cout << "\n|-50000|: " << vabs(b);
    cout << "\n|-6.7|: " << vabs(-6.7);
    system("Pause"); return 0;
}

int vabs(int n) {
    return(n < 0 ? -n : n);
}

long vabs(long n) {
    if (n < 0) return (-n);
    else return n;
}

double vabs(double n) {
    if (n < 0) n = -n;
    return n;
}
```

Pantalla:

```
|-6|: 6
|-50000|: 50000
|-6.7|: 6.7
```

```
#include <iostream>
#include <cstring>
using namespace std;

void copiar(char copia[20],
             const char original[20]) {
    strcpy(copia, original);
}

void copiar(char copia[20],
             const char original[20],
             int n) {
    strncpy(copia, original, n);
}

int main() {
    char cad_a[20], cad_b[20];
    copiar(cad_a, "Buen Día");
    copiar(cad_b, "Buen Día", 4);
    cad_b[4] = '\0';
    cout << cad_a << " y " << cad_b;
    system("Pause");
    return 0;
}
```

Pantalla:

```
Buen Dia y Buen
```

5. Sobrecarga de Métodos y Operadores

5.2 Sobrecarga de métodos (funciones miembros)

- C++ permite definir varios métodos (funciones miembros) con un mismo nombre, **siempre y cuando** difieran en el número y/o el tipo de sus argumentos
- El número y/o tipo de los argumentos de la llamada indicará el método a usar.
- También permite tener dos versiones para un método, una **const** y la otra no, es decir, **permite tener dos métodos idénticos** (mismo nombre, número y tipo de argumentos) **siempre y cuando** una sea un **método constante y el otro no**
- El objeto que realiza la llamada determinará el método a usar:
 - ☐ si el objeto que realiza la llamada es un **objeto constante (const)** se **ejecutará el método constante (const)**
 - ☐ si el **objeto** que invoca el método **no es const**, **ejecutará el método no const**

Ejemplo:

```
#include <cstdlib>
#include <iostream>

using namespace std;

class Punto {
    int x, y;
public:
    Punto(int a, int b) { x = a; y = b; }
    Punto() { x = 0; y = 0; }

    void set(int a, int b) { x=a; y=b; }

    int pr() const { return 0; }
    int pr() { return 1; }
    Punto suma(int n) const;
    Punto suma(int a, int b) const;

    int getx() const { return x; }
    int gety() const { return y; }

    operator char*() const {
        char salida[30];
        sprintf(salida, "(%i,%i)",x,y);
        return strdup( salida );
    }
};

Punto Punto::suma(int n) const {
    Punto p;
    p.set(x+n,y);
    return p;
}

Punto Punto::suma(int a, int b) const {
    return Punto(x+a, y+b);
}
```

```
int main(int argc, char *argv[]) {
    Punto p1(1,2), p2;
    const Punto p3(5,4);
    cout << p1 << ":" << p2 << ":" << p3 << "\n";
    cout << p1.pr() << "\n";
    cout << p3.pr() << "\n";
    p1.set(5,p1.gety());
    //p3.set(5,2); //ERROR p3 es const y set no
    p1=p1.suma(10);
    p2=p3.suma(1,2);
    //p3=p3.suma(10); //ERROR p3 es const y = no
    cout << p1 << ":" << p2 << ":" << p3 << "\n";
    system("PAUSE");
    return EXIT_SUCCESS;
}
```

Pantalla:

```
(1,2):(0,0):(5,4)
1
0
(15,2):(6,6):(5,4)
```

5. Sobrecarga de Métodos y Operadores

5.3 Valores por defecto en los argumentos (argumentos implícitos)

- Son asignados cuando **se omiten** los parámetros en la **llamada**.
- El **valor por defecto** se pone junto al parámetro **tras el signo =**
- Los **argumentos implícitos**:
 - ☐ **deben estar al final** de la lista
 - ☐ Si se omite uno en la llamada, **deben omitirse los que sigan**
 - ☐ Los valores por defecto **deben ser constantes o variables globales**.
 - ☐ **sólo deben indicarse una vez**: en el prototipo de la función o método.
 - ☐ Es una forma de sobrecarga

Ejemplo:

```
float f(int x, int y = 1,
        int z = 0);
...
int main() {
    float v;
    v = f(10);          // f(10, 1, 0)
    v = f(10, 7);       // f(10, 7, 0)
    v = f(10, 2, 3);
    ...
}
```

Pantalla:

3,2
0,0
1,0
5,2
4,2
5,2
7,5
9,4

Ejemplos:

```
...
int perimetro(int b, int h) {
    return(b * 2 + h * 2);
}

int perimetro(int b) {
    return(b * 4);
}

int main() {
    perimetro(6, 4);
    perimetro(8);
    ...
}

// versión argumentos implícitos
int perimetro(int b, int h = 0) {
    if (h == 0)
        h = b;
    return (b * 2 + h * 2);
}
```

```
// versión argumentos implícitos
class Punto {
    int x,y;
public:
    Punto(int a=0, int b=0) { x = a; y = b; }
    Punto add(Punto p, Punto q=0);
    void inc23(Punto p=Punto(2,3))
        { x+=p.x; y+=p.y; }
    void ver() const { cout << x << ", " << y; }
};

Punto Punto::add(Punto p, Punto q) {
    x=x+p.x+q.x; y=p.y+q.y; return (*this);
}
```

```
#include <cstdlib>
#include <iostream>

using namespace std;

class Punto {
    int x,y;
public:
    Punto() { x = 0; y = 0; }
    Punto(int a) { x = a; y = 0; }
    Punto(int a, int b) { x = a; y = b; }
    Punto add(Punto p) {
        x+=p.x; y+=p.y; return (*this);
    }
    Punto add(Punto p, Punto q);
    void inc23() { x+=2; y+=3; }
    void inc23(Punto p) { x+=p.x; y+=p.y; }
    void ver() const { cout << x << ", " << y; }
};

Punto Punto::add(Punto p, Punto q) {
    x=x+p.x+q.x; y=p.y+q.y; return (*this);
}

int main() {
    Punto x(3,2), y, z=1; //z=Punto(1);
    x.ver(); y.ver(); z.ver();
    y.add(x, z);
    x=z.add(y);
    x.ver(); y.ver(); z.ver();
    x.inc23(); y.inc23(z);
    x.ver(); y.ver();
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Nota:

- ☐ La inicialización **z=1** y la asignación del valor por defecto del parámetro **q=0** es posible porque:
 1. existe un constructor con 1 parámetro
 2. El compilador realiza un casting **int --> Punto** invocando el constructor con 1 argumento

5. Sobrecarga de Métodos y Operadores

5.4 Sobrecarga de operadores

- Los operadores de C++ (excepto `::`, `*`, `?`) pueden ser sobrecargados (overloaded), de forma que se puedan utilizar con los objetos de las nuevas clases creadas de la misma forma que se usan con los tipos predefinidos.
- Cuando se sobrecarga un operador:
 - ☐ No se puede cambiar su precedencia y asociatividad.
 - ☐ No puede modificarse el número de operandos que tiene.
 - ☐ No se permite definir nuevos operadores
- En una clase, un mismo operador puede estar sobrecargado varias veces siempre y cuando difiera en el tipo de sus argumentos.
- El tipo del argumento/s en la llamada determinará el método a usar con ese operador.
- Un operador puede sobrecargarse mediante:
 - ☐ Un método de la clase (preferible, por si usamos herencia).
 - ☐ Una función (por eficiencia suele declararse amiga **friend** de la clase)siempre y cuando el operando izquierdo sea de la clase para la que se sobrecarga
- Cuando el operando izquierdo no es un objeto de la clase para la que se sobrecarga, dicho operador únicamente puede sobrecargarse mediante una función.

Al sobrecargar un operador ...

- Relacional o lógico: debe devolver un entero o booleano (**bool**) que indique verdad o falso
- Asignación: `operator=()` debe devolver el objeto para permitir cadenas de asignaciones
`o1 = o2 = o3;`
- Aritmético: debe devolver un objeto de la clase, para permitir expresiones aritméticas complejas y/o para que el resultado se pueda asignar a otro objeto
 - ☐ `operator+()` debe devolver un objeto para permitir cadenas de `+`: (idem con `-`, `*`, `/`)
`o3=o1+o2+o1;` `o3=-o1;` `o3=o1-o2-o3;` `o3=o1*o2*o3;` `o3=o1/o2/o3;` `c=a+b-c*d/a;`
 - ☐ `operator++()` debe devolver el objeto que incrementa para poder hacer:
`obj2 = obj1++;` `obj2 = ++obj1;`
 - Si `++` precede al operando (`++op`), se usa la función `operator++()`. Idem con `--`
 - Si `++` sigue al operando (`op++`), se usa la función `operator++(int notused)`.
- El operando derecho puede ser de otra clase:
`claseA operator+ (int y);` `claseA operator+ (claseA x);` `claseA operator+(claseB y);`
`claseA operator+ (int x, claseA y);` `claseA operator+ (claseB x, claseA y);`
- Si un operador devuelve algo que pueda ser destino, debe retornar una referencia
Ej: `char & cadena::operator[] (int i);` ← **permite hacer** → `cadena x("Cola"); x[3]='t';`
`char & cadena::set(int i);` ← **permite hacer** → `cadena x("Cola"); x.set(3)='t';`

5. Sobrecarga de Métodos y Operadores

5.4.1 Sobrecarga de operadores mediante un método (función miembro)

- El primer operando debe ser un objeto de esa clase (constituye el **argumento implícito**)
- Sintaxis:

```
tipo nombre_clase::operator#([tipo operador_derecho]) {  
    [declaraciones;]  
    sentencias;  
    return valor_devuelto;  
}
```

donde # representa el operador a sobrecargar (+, -, *, /, >, <, >=, <=, ==, !=, etc.)
(operator+, operator-, operator*, operator/, operator>, operator!=, etc.)

- Si un método sobrecarga un operador:
 - ☐ **binario**: el operando izq se pasa implícitamente y el derecho se pasa como argumento
 - ☐ **unario**: el operando se pasa implícitamente (no hay parámetros).
- Dada una clase para la que se ha sobrecargado el operador * y -, la expresión

a * b (operador binario)

se interpreta como

a.operator*(b):

-b (operador unario cambio de signo)

se interpreta como

b.operator-():

5.4.2 Sobrecarga de operadores mediante una función (no miembro)

- Es la única forma de realizar la sobrecarga de un operador cuando el primer operando no es un objeto de la clase para la que se sobrecarga
- Por eficiencia la función se suele declarar amiga (**friend**) de la clase, pero no es obligatorio
- Sintaxis:

```
tipo operator#(tipo operador_izq [, tipo operador_dcho]) {  
    [declaraciones;]  
    sentencias;  
    return valor_devuelto;  
}
```

donde # representa el operador a sobrecargar (+, -, *, /, >, <, >=, <=, ==, !=, etc.)
(operator+, operator-, operator*, operator/, operator>, operator!=, etc.)

- Si una función sobrecarga un operador:
 - ☐ **binario**: hay que pasar explícitamente el operando izquierdo y derecho.
 - ☐ **unario**: hay que pasar explícitamente el operando.
- Dada una clase en la que se ha sobrecargado la suma y el cambio de signo, la expresión

a + b (operador binario)

se interpreta como

operator+(a, b):

-b (operador unario)

se interpreta como

operator-(b):

Ejemplo:

Implementar una clase fracción que permita operar con fracciones

Las operaciones que debe ser capaz de hacer son:

- **Sumar y restar fracciones y enteros.** Ej: $x=2/3+4/2-5/2$, $x=2/3+7$, $x=7+2/3$, $x=2/3-7-3/2$
- **Cambiar el signo de una fracción (-):** Ej: $x=-2/3$, $x=-4$ (x se convierte en $-4/1$)
- **Incrementar la fracción (prefijo y posfijo):** Ej: Si $y=2/3$ $x=++y$ hace que $x=5/3$ $y=5/3$
Ej: Si $y=2/3$ $x=y++$ hace que $x=2/3$ $y=5/3$
- **Comparar 2 fracciones (> y ==):** Ej: Si $x=4/6$ $y=2/3$ $x==y$ es true $x>y$ es false

Implementa un programa que genere la siguiente salida a partir de a(3,2), b(3), c(2,3) y d:

	a	b	c	d
	3/2	3/1	2/3	0/1
a=a+a	6/2	3/1	2/3	0/1
b=a-c	6/2	14/6	2/3	0/1
a=-b	-14/6	14/6	2/3	0/1
d=1+a-c+b	-14/6	14/6	2/3	2/6
a++b	20/6	20/6	2/3	2/6
a=c++	2/3	20/6	5/3	2/6
a=b+5	50/6	20/6	5/3	2/6
c=5+b	50/6	20/6	50/6	2/6
a=a>b?-d:b	-2/6	20/6	50/6	2/6
a y c distintos				

Solución:

Sobrecarga con métodos	Sobrecarga con función (friend)
<pre>#include <iostream> //cin, cout using namespace std; int mcm(int a, int b) { int comun, mayor; if (a<0) a=-a; if (b<0) b=-b; mayor = a>b ? a : b; for (comun=mayor; ; comun+=mayor) if (comun%a==0 && comun%b==0) return comun; } class frac { int n, d; public: frac() { n=0; d=1; } frac(int n, int d=1) { set(n,d); } int getn() const { return n; } int getd() const { return d; } void set(int n, int d=1); frac operator+(frac q) const; frac operator-(frac q) const; frac operator-() const; frac operator++(); frac operator++(int flag); int operator==(frac q) const; bool operator>(frac q) const; //operator+(int,frac) no lo declaro amiga void ver() const; //signo en numerador };</pre>	<pre>#include <iostream> //cin, cout using namespace std; int mcm(int a, int b) { int comun, mayor; if (a<0) a=-a; if (b<0) b=-b; mayor = a>b ? a : b; for (comun=mayor; ; comun+=mayor) if (comun%a==0 && comun%b==0) return comun; } class frac { int n, d; public: frac() { n=0; d=1; } frac(int n, int d=1) { set(n,d); } int getn() const { return n; } int getd() const { return d; } void set(int n, int d=1); friend frac operator+(frac p, frac q); friend frac operator-(frac p, frac q); friend frac operator-(frac p); friend frac operator++(frac &p); friend frac operator++(frac &p, int flag); friend int operator==(frac p, frac q); friend bool operator>(frac p, frac q); //friend frac operator+(int n, frac q); void ver() const; //signo en numerador };</pre>

```

void frac::set(int n, int d) {
    if (d==0) exit(1);
    this->n=n; this->d=d;
}

frac frac::operator+(frac q) const {
    frac suma;
    int comun = mcm(d, q.d);
    suma.d = comun;
    suma.n = n*comun/d + q.n*comun/q.d;
    return suma;
}

frac frac::operator-(frac q) const {
    frac menosq(-q.n, q.d);
    return (*this)+menosq;
}

frac frac::operator-() const {
    return frac(-n, d);
}

frac frac::operator++() { // ++obj
    n+=d;
    return *this; //devuelve una copia del
} //objeto que invoca el ++

frac frac::operator++(int flag) { //obj++
    frac copia(*this);
    n+=d;
    return copia;
}

frac operator+(int n, frac q) const {
    frac suma(n*q.getd()+q.getn(), q.getd());
    return suma;
}

int frac::operator==(frac q) const {
    if (n*q.d==d*q.n)
        return 1;
    return 0;
}

bool frac::operator>(frac q) const {
    return (n*q.d>d*q.n);
}

void frac::ver() const {
    if (d>0) cout << n << "/" << d << "\t";
    else cout << -n << "/" << -d << "\t";
}

int main() {
    frac a(3,2),b(3),T[]={frac(2,3),frac()};
    cout << "\n\t a\tb\tc\td\n\t ";
    a.ver();b.ver();T[0].ver();T[1].ver();//(*)
    a=a+a; cout << "\na=a+a\t "; //(*)
    b=a-T[0]; cout << "\nb=a-c\t "; //(*)
    a=-b; cout << "\na=-b\t "; //(*)
    T[1]=1+a-T[0]+b; cout << "\nd=1+a-c+b "; //(*)
    a+=b; cout << "\na+=b\t "; //(*)
    a=T[0]++; cout << "\na=c++\t "; //(*)
    a=b+5; /*(1)*/cout << "\na=b+5\t "; //(*)
    T[0]=5+b; cout << "\nc=5+b\t "; //(*)
    a=a>b?-T[1]:b; cout << "\na=a>b?-d:b"; //(*)
    if (a==T[0]) cout << "\na y c iguales\n";
    else cout << "\na y c distintos\n";
    system("PAUSE"); return EXIT_SUCCESS;
}

```

Las funciones no tienen const

```

void frac::set(int n, int d) {
    if (d==0) exit(1);
    this->n=n; this->d=d;
}

frac operator+(frac p, frac q) {
    frac suma;
    int comun = mcm(p.d, q.d);
    suma.d = comun;
    suma.n = p.n*comun/p.d + q.n*comun/q.d;
    return suma;
}

frac operator-(frac p, frac q) {
    frac menosq(-q.n, q.d);
    return p+menosq;
}

frac operator-(frac p) {
    return frac(-p.n, p.d);
}

frac operator++(frac &p) { // ++obj
    p.n+=p.d;
    return p;
}

frac operator++(frac &p, int flag) {
    frac copia(p);
    p.n=p.n+p.d;
    return copia;
}

/*frac operator+(int n, frac q) {
    frac suma(n*q.d+q.n, q.d);
    return suma;
}*/

int operator==(frac p, frac q) {
    if (p.n*q.d==p.d*q.n)
        return 1;
    return 0;
}

bool operator>(frac p, frac q) {
    return (p.n*q.d>p.d*q.n);
}

void frac::ver() const {
    if (d>0) cout << n << "/" << d << "\t";
    else cout << -n << "/" << -d << "\t";
}

int main() {
    frac a(3,2),b(3),T[]={frac(2,3),frac()};
    cout << "\n\t a\tb\tc\td\n\t ";
    a.ver();b.ver();T[0].ver();T[1].ver();//(*)
    a=a+a; cout << "\na=a+a\t "; //(*)
    b=a-T[0]; cout << "\nb=a-c\t "; //(*)
    a=-b; cout << "\na=-b\t "; //(*)
    T[1]=1+a-T[0]+b; cout << "\nd=1+a-c+b "; //(*)
    a+=b; cout << "\na+=b\t "; //(*)
    a=T[0]++; cout << "\na=c++\t "; //(*)
    a=b+5; /*(1)*/cout << "\na=b+5\t "; //(*)
    T[0]=5+b; cout << "\nc=5+b\t "; //(*)
    a=a>b?-T[1]:b; cout << "\na=a>b?-d:b"; //(*)
    if (a==T[0]) cout << "\na y c iguales\n";
    else cout << "\na y c distintos\n";
    system("PAUSE"); return EXIT_SUCCESS;
}

```

Observaciones:

- La suma de un entero con una fracción (**int+frac**) no la podemos codificar con un método ya que el operando izquierdo (el implícito) no es un objeto **frac** sino un **int**, por tanto hay que codificarlo con una función no miembro (**friend** o no, según queramos).

```
frac operator+(int n, frac q);
```

- La suma de una fracción con un entero (**frac+int**) no la hemos codificado (hemos codificado (**frac+frac**) y (**int+frac**)). ¿Cómo es posible que funcione?...

... porque el compilador hace un **casting implícito** convirtiendo el entero en una fracción, gracias a que existe un constructor que admite un parámetro.

```
a=b+5; //el compilador lo convierte en a=b+frac(5);
```

Si no hubiera un constructor con un parámetro o la suma de $x/y + z$ tuviera que realizar una operación diferente a $x/y + z/1$ entonces tendríamos que sobrecarga la operación.

```
frac frac::operator+(int n); o bien frac operator+(frac p, int n);
```

- Pero entonces... ¿el compilador no puede hacer un **casting implícito** con la suma de un entero con una fracción (**int+frac**), al igual que hace con la suma de un (**frac+int**)?

```
a=5+b; //el compilador lo convierte en a=frac(5)+b;
```

- ☐ La respuesta es **sí**, por eso en la versión en la que hemos sobrecargado los operadores con funciones amigas (parte derecha), la sobrecarga de (**int+frac**) no está implementada (**se indica como habría que hacerlo, pero está comentada entre /* y */**)
- ☐ En la sobrecarga de operadores con métodos (parte izquierda), la sobrecarga de (**int+frac**) **SI** está implementada ya que es **obligatoria (al implementarse con un método el operando izquierdo a la fuerza debe ser un objeto de la clase)**.

- Para mejorar la eficiencia los parámetros por valor podríamos haberlo pasado mediante **referencias constantes** (así no se tiene que hacer copia de los parámetros)

<pre>class frac { int n, d; public: frac() { n=0; d=1; } frac(int n, int d=1) { set(n,d); } int getn() const { return n; } int getd() const { return d; } void set(int n, int d=1); frac operator+(const frac &q) const; frac operator-(const frac &q) const; frac operator-() const; frac operator++(); frac operator++(int flag); int operator==(const frac &q) const; bool operator>(const frac &q) const; //operator+(int, const frac &q) void ver() const; };</pre>	<pre>class frac { int n, d; public: frac() { n=0; d=1; } frac(int n, int d=1) { set(n,d); } int getn() const { return n; } int getd() const { return d; } void set(int n, int d=1); friend frac operator+(const frac &p, const frac &q); friend frac operator-(const frac &p, const frac &q); friend frac operator-(const frac &p); friend frac operator++(frac &p); friend frac operator++(frac &p, int flag); friend int operator==(const frac &p, const frac &q); friend bool operator>(const frac &p, const frac &q); //friend frac operator+(int n, const frac &q); void ver() const; };</pre>
---	--

- A pesar de no sobrecargar el operador = lo hemos podido usar porque el compilador (en caso de no existir sobrecarga) crea un operador de asignación por defecto que hace una copia binaria (sólo lo tendríamos que redefinir cuando la copia binaria no fuera lo correcto)

Observaciones: (continuación)

- Para **mejorar la eficiencia** aún más los operadores sobrecargados que modifican el propio objeto podíamos haberlo devuelto mediante referencias (devolvería el propio objeto en lugar de una copia del objeto). En nuestro ejemplo el único operador que devuelve el propio objeto modificado es el **operator++ prefijo** (++x)

<pre>class frac { int n, d; public: ... frac& operator++(); frac operator++(int flag); };</pre>	<pre>class frac { int n, d; public: ... friend frac& operator++(frac &p); friend frac operator++(frac &p, int flag); };</pre>
--	--

No podemos aplicarlo al ++ postfijo ya que devuelve copia que es un objeto local

<pre>frac& frac::operator++() { //++obj n++; return *this; } frac frac::operator++(int flag){ //obj++ frac copia(*this); n++; return copia; }</pre>	<pre>frac& operator++(frac &p) { // ++obj p.n++; return p; } frac operator++(frac &p, int flag) { frac copia(p); p.n++; return copia; }</pre>
--	--

- La sobrecarga de operadores se podía hacer sin funciones amigas (**friend**). La modificación de los atributos privados se haría a través de la interfaz pública de la clase.

<pre>class frac; //declaracion anticipada frac operator+(const frac &p, const frac &q); frac operator-(const frac &p, const frac &q); frac operator-(const frac &p); frac operator++(frac &p); frac operator++(frac &p, int flag); int operator==(const frac &p, const frac &q); bool operator>(const frac &p, const frac &q); //frac operator+(int n, const frac &q); class frac { int n, d; public: frac() { n=0; d=1; } frac(int n, int d=1) { set(n,d); } int getn() const { return n; } int getd() const { return d; } void set(int n, int d=1); void ver() const; //signo en numerador }; void frac::set(int n, int d) { if (d==0) exit(1); this->n=n; this->d=d; } frac operator+(const frac &p, const frac &q) { frac suma; int comun = mcm(p.getd(), q.getd()); suma.set(p.getn()*comun/p.getd()+ q.getn()*comun/q.getd(), comun); return suma; } frac operator-(const frac &p) { return frac(-p.getn(), p.getd()); }</pre>	<pre>frac operator-(const frac &p, const frac &q) { frac menosq(-q.getn(), q.getd()); return p+menosq; } frac operator++(frac &p) { // ++obj p.set(p.getn()+p.getd(),p.getd()); return p; } frac operator++(frac &p, int flag) { frac copia(p); p.set(p.getn()+p.getd(),p.getd()); return copia; } /*frac operator+(int n, const frac &q) { frac suma(n*q.getd()+q.getn(), q.getd()); return suma; }*/* int operator==(const frac &p, const frac &q) { if (p.getn()*q.getd()==p.getd()*q.getn()) return 1; return 0; } bool operator>(const frac &p, const frac &q) { return (p.getn()*q.getd())>p.getd()*q.getn(); } void frac::ver() const { if (d>0) cout << n << "/" << d << "\t"; else cout << -n << "/" << -d << "\t"; }</pre>
--	--

5. Sobrecarga de Métodos y Operadores

5.5 Sobrecarga de ciertos operadores

5.5.1 Sobrecarga del operador de asignación =

- Si el programador no lo sobrecarga, el compilador lo sobrecarga (realizando una copia binaria de los atributos)
- Sólo se puede sobrecargar mediante un método (función miembro).
- Sintaxis: (permite cadenas de asignaciones y máxima eficiencia)

```
class & clase::operator=(const clase& operador_derecho) ;
```

- void operator=(...) permite hacer a=b, pero no encadenar asignaciones a=b=c;
- class operator=(...) { ... return *this; } permite encadenar asignaciones a=b=c; Devuelve copia del objeto
(x=y=z).metodo(); → x=y=z; copiaux.metodo(); (x=y=z)++ → x=y=z; copiaux++;
- class& operator=(...) { ... return *this; } permite encadenar asignaciones a=b=c; Devuelve objeto original
(x=y=z).metodo(); → x=y=z; x.metodo(); (x=y=z)++ → x=y=z; x++;
- parámetros referencias constantes → para que cuando escribamos cd=cd se detecte con (this != &cd)

- La sobrecarga por defecto que proporciona el compilador hace una copia binaria de los datos:

```
class & clase::operator=(const clase& operador_derecho) {  
    *this=operador_derecho; //llamada recursiva ERROR  
    if (this != &operador_derecho) { //por si es a=a y no es a=b  
        sentencias_que_hacen_una_copia_binaria;  
    }  
    return *this; //con & devuelve el propio objeto implicito  
} //sin & devuelve una copia del objeto implicito
```

- El operador de asignación = lo podemos sobrecargar con otros objetos o tipos de datos:

```
class & clase::operator=(tipo operador_derecho) ;  
class & clase::operator=(const otraClase &operador_derecho) ;
```

El compilador no sobrecarga el operador = para estos casos: el programador lo tiene que hacer explícitamente.

5.5.2 Sobrecarga del operador += (o del operador -=, operador/=:, operador*=)

- Se puede sobrecargar mediante un método o una función no miembro.
- Sintaxis: (permite cadena de += y al ser por referencia máxima eficiencia al no hacer copia)

```
class & clase::operator+=(const clase &operador_derecho) ;  
class & operador+=(clase &op_izq, const clase &op_derecho) ;
```

- El operador de asignación+ = lo podemos sobrecargar con otros objetos o tipos de datos:

```
class & clase::operator+=(tipo operador_derecho) ;  
class & clase::operator+=(const otraClase &operador_derecho) ;
```

```
class & operador+=(clase& op_izq, tipo operador_derecho) ;  
class & operador+=(clase& op_izq, const otraClase& op_derecho) ;
```

El compilador no sobrecarga el operador +=. Para poder usarlo con objetos de nuevas clases creadas, el programador las tiene que sobrecargar explícitamente.

Sobrecarga Operador =

Ejemplo: sobrecargar el operador = y += para poder usarlo en una clase cadena, de forma que permita copiar una cadena en otra utilizando dicho operador

```
#include <iostream> //cin, cout
using namespace std;

class cadena {
    char *s;
    int nchar;
public:
    cadena(char*); // constructor general
    ~cadena() { delete [] s; }
    int getnchar() const { return nchar; }
    const char *gets() const { return s; }
    void sets(char *);
    void ver() const;
    cadena& operator= (const cadena& cd);
    cadena& operator+= (const cadena& cd);
};

cadena::cadena(char* c) {
    nchar = strlen(c);
    s = new char[nchar+1];
    strcpy(s, c);
}

void cadena::sets(char* c) {
    delete [] s;
    nchar = strlen(c);
    s = new char[nchar+1];
    strcpy(s, c);
}

void cadena::ver() const {
    cout << nchar << ", " << s << endl;
}

cadena& cadena::operator=(const cadena& cd) {
    if (this != &cd) { //por si es cd=cd
        nchar = cd.nchar; //para ello cd debe
        delete [] s; //ser por referencia &
        s = new char[nchar + 1];
        strcpy(s, cd.s);
    }
    return *this;
}

cadena& cadena::operator+=(const cadena& cd) {
    char *aux=s;
    nchar += cd.nchar;
    s = new char[nchar + 1];
    strcpy(s, aux);
    strcat(s, cd.s);
    delete [] aux;
    return *this;
}

int main() {
    cadena c1("toro");
    cadena c2("cobra");
    cout << "c1:"; c1.ver();
    cout << "c2:"; c2.ver();
    c2=c1;
    cout << "c2:"; c2.ver();
    c1.sets("sentado");
    cout << "c1:"; c1.ver();
    cout << "c2:"; c2.ver();
    c2+=c1;
    cout << "c1:"; c1.ver();
    cout << "c2:"; c2.ver();
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Pantalla:

```
c1:4,toro
c2:5,cobra
c2:4,toro
c1:7,sentado
c2:4,toro
c1:7,sentado
c2:11,torosentado
```

- ✓ Si no sobrecargamos el operador =, el compilador al realizar la asignación

c2=c1

haría una copia binaria del objeto c1 en c2, con lo que ambos objetos apuntarían a la misma dirección de memoria (el valor de s es el mismo en ambos), en lugar de producir una copia.

Como consecuencia de ello, ambos compartirían la misma cadena, de forma que los cambios provocados en uno de ellos afectarían al otro, al apuntar ambos a la misma zona de memoria.

- ✓ Si no sobrecargamos el operador += el compilador daría un error al encontrar la instrucción

c2+=c1

ya que dicho operador por defecto sólo se puede utilizar con los tipos básico de C++ (int, float, char) y para poderlo usar con las nuevas clases creadas es necesario sobrecargarlo.

Pruebe a eliminar la sobrecarga del operador = y vea lo que ocurre

A continuación, haga lo mismo eliminando la sobrecarga del operador +=

Nota: la sentencia if en la sobrecarga de operador de asignación evita la asignación de un objeto a sí mismo (cd=cd) ya que, en ese caso, al liberar la memoria del 1er operando, el 2º (que es el mismo) la pierde también, con lo que habría un error. Para ello **el operando derecho hay que pasarlo por referencia** para pasar el propio objeto, de forma que detecte que ambos objetos **this** (dir objeto izquierdo) y **&cd** (dir del objeto derecho) son el mismo. Si se pasa por valor o copia, &cd devuelve la dir de la copia que no es la misma que la del objeto original.

5. Sobrecarga de Métodos y Operadores

5.5.3 Sobrecarga del operador []

- Se puede sobrecargar mediante un método o una función no miembro.
- Sintaxis: (al devolver una referencia permite modificar lo que se devuelve)

```
tipo1 & clase::operator[](tipo2 i); // miembro
tipo1 & operator[](const clase &op_izq, tipo2 i); //no miembro
```

siendo **tipo1** el tipo indexado y **tipo2** el índice (puede ser cualquier tipo, lo normal es **int**) y donde **&** es opcional, dependiendo de si nos interesa o no que devuelva una referencia a un objeto **tipo1** o que devuelva un objeto **tipo1**.

- Si devuelve una referencia, entonces el operador [] **podrá utilizarse** a la derecha, pero también a la izquierda de una asignación, con el peligro que eso conlleva si la clase tiene alguna restricción respecto a los valores de sus atributos (véase ejemplo apartado 4.4)
- Al devolver una referencia permite modificar lo que se devuelve

```
clase x; tipo1 y,z; int i;
//tipo2 es int
...
x[2]=y; //x.operator[](2)=y
z=x[i];
```

```
clase x; tipo1 y,z; char *cad;
//tipo2 es char *
...
x["ana"]=y; //x.operator[]("ana")=y
z=x[cad];
```

Si no queremos permitir que `operator[]` pueda aparecer en la parte izquierda de una asignación, haremos que no devuelva una referencia o que la referencia sea **const**.

Ejemplo: sobrecargar el operador [] para poder usarlo en una clase cadena, de forma que permita acceder y/modificar los caracteres indexándolos a partir del 1

```
#include <iostream> //cin, cout
using namespace std;
class cadena {
    char *s;
    int nchar;
public:
    cadena(char*); // constructor general
    ~cadena() { delete [] s; }
    int getnchar() const { return nchar; }
    char& operator[](int i);
    const char * gets() const { return s; }
};

cadena::cadena(char* c) {
    nchar = strlen(c);
    s = new char[nchar+1];
    strcpy(s, c);
}

char& cadena::operator[](int i) {
    if (i <= nchar && i>=1)
        return s[i-1];
    else {
        cout << "ERROR\n";
        system("PAUSE");
        exit(1); //aborta el programa
    }
}
```

```
int main() {
    cadena cad("casa");
    char h;
    cout << cad.gets() << " tiene " <<
        cad.getnchar() << " letras\n";
    cad[3] = 'n'; //cad.operator[](3)='n';
    cout << "Introduzca una letra: ";
    cin >> cad[2]; //cin>>cad.operator[](2);
    cout << cad.gets() << endl;
    cout << "la 1ª letra es " << cad[1] << endl;
    h = cad[4]; //h=cad.operator[](4);
    cout << "la 4ª letra es " << h << endl;
    cad[9] = 'k'; //ERROR 9 excede cad
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Pantalla:

```
casa tiene 4 letras
Introduzca una letra: e
cena
la primera letra es c
la cuarta letra es a
ERROR
```

5. Sobrecarga de Métodos y Operadores

5.5.4 Sobrecarga de los operadores de inserción << y de extracción >>

- Los operadores de **inserción (<<)** y **extracción (>>)** en los flujos (streams) de E/S sólo se pueden sobrecargar mediante una función no miembro.
- Sintaxis: (al devolver una referencia permite modificar lo que se devuelve)

```
friend std::istream& operator>>(std::istream& s, clase &o);  
friend std::ostream& operator<<(std::ostream& s, const clase &o);
```

donde **istream** es el *flujo* de entrada y **ostream** el *stream* de salida y lo gris es opcional

Estos **flujos** funcionan como cintas transportadoras que entran (>>) o salen (<<) del programa. Se recibe una **referencia al flujo** como 1er argumento, se añade o se retira de él **la variable que se desee**, y se devuelve siempre como valor de retorno **una referencia al flujo** (stream) modificado.

- **istream** y **ostream** se devuelven por referencia para permitir cadenas de << y de >>
o1 << o2 << o3; o1 >> o2 >> o3;
- ambas se suelen declarar **amigas** de la clase para la que se sobrecarga, para poder acceder a su parte privada, aunque no es obligatorio
- en **operator<<** el objeto de la clase para la que se sobrecarga se suele pasar por **referencia** (por motivos de eficiencia) **constante** (por seguridad), aunque no es obligatorio.

El C++ no tiene instrucciones de E/S, pero si tiene librerías que manejan **streams** (flujos). La librería **<iostream>** proporciona una serie de clases que permiten la E/S por teclado (**istream**) como por pantalla (**ostream**).

El C++ tiene predefinidos los objetos streams **cin** y **cout**:

- **cin** es un objeto de la clase **istream** que se encarga de la entrada por teclado
- **cout** es un objeto **ostream** que maneja la salida por pantalla.
- << y >> son operadores que están sobrecargados para las clases de streams

Ejemplo: sobrecarga de >> con función amiga y << con función no amiga

```
#include <iostream> //cin, cout
```

```
using namespace std;
```

```
class frac {
```

```
    int n, d;
```

```
public:
```

```
    frac() { n=0; d=1; }
```

```
    frac(int x, int y) { n=x; d=y; }
```

```
    int getn() const { return n; }
```

```
    int getd() const { return d; }
```

```
    void set(int n, int d=1);
```

```
    friend istream& operator>>(istream& s,  
                               frac &p);
```

```
};
```

```
istream& operator>>(istream& s, frac &p) {
```

```
    cout << "numerador: ";
```

```
    s >> p.n; //s actúa a modo de cin
```

```
    do {
```

```
        cout << "denominador: ";
```

```
        s >> p.d; //s actúa a modo de cin
```

```
    } while (p.d==0);
```

```
    return s;
```

```
}
```

```
ostream& operator<<(ostream &s,
```

```
                    const frac &p) {
```

```
    if (p.getd()>0)
```

```
        s << p.getn() << "/" << p.getd();
```

```
    else //s actúa a modo de cout
```

```
        s << -p.getn() << "/" << -p.getd();
```

```
    return s;
```

```
}
```

```
int main() {
```

```
    frac a(3,2), b;
```

```
    cout << a << " , " << b << endl;
```

```
    cout << "Introduce 2 fracciones \n";
```

```
    cin >> a >> b;
```

```
    cout << a << " , " << b << endl;
```

```
    system("PAUSE"); return EXIT_SUCCESS;
```

```
}
```

Si operator>> no fuera amiga:

```
istream& operator>>
```

```
(istream& s, fracc &p) {
```

```
    int n, d;
```

```
    cout << "numerador: ";
```

```
    s >> n;
```

```
    do {
```

```
        cout << "denominador: ";
```

```
        s >> d;
```

```
    } while (d==0);
```

```
    p.set(n,d);
```

```
    return s;
```

```
}
```

Pantalla:

3/2 , 0/1

Introduce 2 fracciones

numerador: 2

denominador: 3

numerador: 2

denominador: 0

denominador: 5

2/3 , 2/5

Fácil de programar:

- en **operator>>**, s actúa como cin
- en **operator<<**, s actúa como cout

5. Sobrecarga de Métodos y Operadores

5.5.5 Operadores de conversión de tipos vs constructores de conversión

- Cuando definimos clases (son nuevos tipos) el compilador no define ninguna forma de conversión a otras clases o tipos estándar. El programador debe indicar como se hace.
- Estas conversiones pueden ser realizadas de 2 formas, (en un sentido u otro) mediante:
 - **constructores de conversión:** permiten convertir el objeto o el tipo estándar pasado como parámetro, en un objeto de la clase que lo sobrecarga (*sentido desde*)
 - **operadores de conversión:** permiten convertir un objeto de la clase que lo sobrecarga, a un objeto de otra clase o a un tipo estándar (*sentido hacia*)

5.5.5.1 Sobrecarga de constructores de conversión

- Sintaxis: (la 1ª convierte otraClase en clase, la 2ª convierte tipo_estandar en clase)

```
class::class(const otraClase &o); //conversión desde otra clase
class::class(tipo_estandar o);    //conversión desde un tipo
```

donde **const &** es opcional (& por motivos de eficiencia, **const** por motivos de seguridad)

5.5.5.2 Sobrecarga de operadores de conversión de tipos

- Los operadores de conversión (**cast**) convierten objetos de una clase en objetos de otra
- Sintaxis: (la 1ª convierte clase en otraClase, la 2ª convierte clase en tipo_estandar)

```
class::operator otraClase() const; //convierte clase en otraClase
class::operator tipo_estandar() const; //conversión de clase a tipo_estandar
```

- Sólo se pueden sobrecargar mediante un método sin argumentos ni valor de retorno, ni siquiera **void** (esto no significa que no devuelvan nada, sino que no hay que indicarlo al ser el tipo de retorno el propio tipo al que se quiere convertir).

Ejemplo:

```
#include <iostream>
using namespace std;
class tiempo {
    int hora, minuto;
public:
    tiempo(int h, int m);
    tiempo(int m);
    operator int() const {return hora*60+minuto;}
    friend ostream& operator<<(ostream &s,
                                const tiempo &t);
};
tiempo::tiempo(int h, int m) {
    hora=h+m/60;
    minuto=m%60;
}
tiempo::tiempo(int m) {
    hora=0; minuto=m;
    while(minuto >= 60) {
        minuto -= 60;
        hora++;
    }
}
```

Pantalla:

```
127, 1:30
0:30, 90
120
2:0, 120
```

```
ostream& operator<<(ostream &s, const tiempo &t) {
    s << t.hora << ":" << t.minuto;
    return s; //s actúa a modo de cout
}

int main() {
    tiempo a(2,7), b(90), c(0,0);
    int m;
    cout << (int)a << " , " << b << endl;
    //a+=15; //ERROR += no esta sobrecargado
    a=15; //a=tiempo(15)
    a=a+15; //a=tiempo((int)a+15)
    cout << a << " , " << (int)b << endl;
    m = a; //m=(int)a;
    m+= b; //m+=(int)b;
    cout << m << endl;
    m = c = a+b;
    //c=tiempo((int)a+(int)b); m=(int)c;
    cout << c << " , " << m << endl;
    system("PAUSE"); return EXIT_SUCCESS;
}
```

5. Sobrecarga de Métodos y Operadores

5.5.5 Operadores de conversión de tipos vs constructores de conversión

Ejemplo 2: conversión de tipos y clases vs constructores de conversión

```
#include <iostream>
using namespace std;

class reloj {
    int horas, minutos;
public:
    reloj(int h, int m);
    reloj operator+(int m);
    int geth() const { return horas; }
    int getm() const { return minutos; }
};

reloj::reloj(int h, int m) {
    minutos=m%60;
    horas=(h+m/60)%24;
}

reloj reloj::operator+(int m) {
    reloj suma(horas,minutos+m);
    return suma;
}

class tiempo {
    int hora, minuto;
public:
    tiempo(int h, int m);
    tiempo(int m);
    tiempo(const reloj &r);
    operator int() const;
    operator float() const;
    operator reloj() const;
    tiempo operator-(tiempo t) const;
    friend ostream& operator<<(ostream &s,
                                const tiempo &t);
};

tiempo::tiempo(int h, int m) {
    hora=h+m/60;
    minuto=m%60;
}

tiempo::tiempo(int m) {
    hora=m/60;
    minuto=m%60;
}

tiempo::tiempo(const reloj &r) {
    hora=r.geth();
    minuto=r.getm();
}

tiempo::operator int() const {
    return hora*60+minuto;
}

tiempo::operator float() const {
    return hora+(float)minuto/60;
}
```

Pantalla:

```
2h2m
1h31m
26:2(1562), 1.5
0:30, 1:30
120
-60, -1:0
-60, -1:0
124. 2:4
```

```
tiempo::operator reloj() const {
    reloj aux(hora,minuto);
    return aux;
}

tiempo tiempo::operator-(tiempo t) const {
    tiempo aux(hora-t.hora, minuto-t.minuto);
    return aux;
}

ostream& operator<<(ostream &s, const tiempo &t) {
    s << t.hora << ":" << t.minuto;
    return s; //s actúa a modo de cout
}

int main() {
    reloj x(25,62), y(0,0);
    tiempo a(25,62), b(90), c(x);
    int minus;
    y=b; //y=(reloj)b;
    y=y+60*24+1; //y=y.operator+(1441);
    cout << x.geth() << "h" << x.getm() << "m\n";
    cout << y.geth() << "h" << y.getm() << "m\n";
    cout << a << "(" << (int)a << "," << (float)b;
    a=15; //a=tiempo(15);
    //a=a+15; //ERROR ambiguo, puede ser
    //a=tiempo((int)a+15)
    //a=tiempo((float)a+15)

    a=(int)a+15;
    //a+=15; //ERROR += no sobrecargado
    cout << a << ", " << b << endl;
    minus = a; //minus=(int)a
    //minus=a+b; //ERROR ambiguo, puede ser
    //minus=(int)a+(int)b
    //minus=(float)a+(int)b
    //minus=(int)a+(float)b
    //minus=(float)a+(float)b

    minus = (int)a + (int)b;
    //minus+= b; //ERROR ambiguo, puede ser
    //minus+=(int)b
    //minus+=(float)b

    cout << minus << endl;
    minus = c = a - b;
    //c = a.operator-(b); minus = (int)c;
    cout << minus << ", " << c << endl;
    c = minus = a - b;
    //minus=(int)a.operator-(b); c=tiempo(minus)
    cout << minus << ", " << c << endl;
    c = x+2;
    //c=tiempo(x.operator+(2));
    cout << (int)c << ", " << c << endl;
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Pruebe a eliminar la sobrecarga del operador – en la clase tiempo y vea lo que ocurre
Modifica el main para que la resta a – b se pueda realizar sin necesidad de sobrecargar el –
Solución: minus = c = (int)a – (int) b; //asi se elimina la ambigüedad (se restan 2 enteros y casting de int a tiempo)

6. Constructores y Destructores

- Una clase debe tener al menos un constructor (puede tener varios) y un único destructor

Constructor

- Es un método que tiene el mismo nombre que la clase en la que está definido y no tiene tipo devuelto (ni void).
- Es llamado automáticamente cada vez que se crea un objeto de esa clase (si el objeto es global: al empezar el programa, si es local: al alcanzar su declaración dentro del bloque donde está definido, si es dinámico, al crearlo con **new**).
- Puede tener argumentos (**con o sin valor por defecto**) y puede estar sobrecargado.
- Se usa para inicializar los valores de los atributos (variables miembros) de la clase.

Destructor

- Idem, pero el nombre viene precedido por el carácter ~ (Alt+126) y no tiene tipo devuelto (ni siquiera void).
- Es llamado automáticamente cuando el objeto va a dejar de existir (si el objeto es global: al terminar el programa, si es local: al salir del bloque donde está definido, si es un objeto dinámico **new**, al destruirlo con **delete**).
- No tiene argumentos y no puede estar sobrecargado (es siempre único).
- Se usa para liberar memoria dinámica, para terminar asuntos pendientes, cerrar ficheros, etc.

- Si el programador no define ningún constructor para una clase, el compilador proporciona el siguiente **constructor de oficio** (default constructor) sin argumentos (que no hace nada):

```
class::class() { //constructor de oficio, sin argumentos
    //no hace nada (lo crea el compilador si no definimos ninguno)
}
```

- Lo mismo ocurre con el destructor. Si el programador no define ningún destructor para una clase, el compilador proporciona el siguiente **destructor de oficio** (no hace nada):

```
class::~~class() { //destructor de oficio
    //no hace nada (lo crea el compilador si no definimos ninguno)
}
```

6.1 Construcción, destrucción y tiempo de vida

- Para **objetos locales**, el constructor se invoca cada vez que se crea el objeto, y su destructor al terminar la función o bloque en el que está.
- Para **objetos globales**, el constructor se invoca una sola vez, al inicio del programa, y el destructor al terminar el programa.
- Para **objetos dinámicos**, el constructor se invoca cuando se crea con **new** y el destructor cuando se destruye con **delete**.
- Los destructores son llamados en orden inverso a los constructores.

Orden de creación y destrucción de los objetos:

1 Creación de x, y	7 Destrucción de p
2 Creación de a	8 Creación de b
3 Creación de b	9 Destrucción de b
4 Destrucción de b	10 Destrucción de d, c
5 Creación de p	11 Destrucción de a
6 Creación de c, d	12 Destrucción de y, x

```
class frac {
public:
    frac(); // constructor
    ~frac(); // destructor
}

frac x,y; //objeto global

void proceso() {
    frac b; // objeto local
    ...
}

int main() {
    frac a,*p; //objeto local
    proceso();
    p = new frac();
    frac c,d; // objeto local
    delete p;
    proceso();
    ...
}
```

6. Constructores y Destructores

6.2 Inicializadores

- Sólo lo pueden usar los constructores, no lo pueden usar otros métodos.
- Los inicializadores se introducen separados por comas, tras el carácter dos puntos (:), justo antes de abrir las llaves del cuerpo del constructor
- Sintaxis:

```
class::class([parámetros]): atributo1(valor1), atributo2(valor2)... {  
    [sentencias;]           //constructor con inicializadores  
}
```

donde **atributoN** son atributos de la clase y **valorN** los valores que queremos asignarles

lo anterior es equivalente a:

```
class::class([parámetros]) {  
    atributo1 = valor1;      //constructor sin inicializadores  
    atributo2 = valor2;  
    [sentencias;]  
}
```

- Los inicializadores son más eficientes que las sentencias de asignación.
- **Son necesarios** cuando en una clase definimos atributos (variables miembros) que son referencias (&) o constantes (**const**), ya que deben inicializarse, al no poder modificar sus valores una vez contruidos (no podemos usar sentencias de asignación).

Ejemplo: clase que tiene atributos constantes y atributos referencia

```
#include <iostream>  
using namespace std;  
class cla {  
    int a,b;  
    int &ref;  
    const float fijo;  
public:  
    cla():a(0),b(0),ref(a),fijo(9.8) { }  
    cla(int x, int y, float v)  
        :a(x),b(y),ref(a),fijo(v) { }  
    void set(int x, int y) { a=x; b=y; }  
};  
int main() {  
    cla a, b(2, 3, 0.5);  
    b.set(6,3);  
    system("PAUSE");return EXIT_SUCCESS;  
}
```

Lo anterior es equivalente a:

```
class cla {  
    int a,b;  
    int &ref;  
    const float fijo;  
public:  
    cla():ref(a),fijo(9.8) { a=0; b=0; }  
    cla(int x, int y, float v);  
    void set(int x, int y) { a=x; b=y; }  
};  
  
cla::cla(int x, int y, float v):ref(a), fijo(v) {  
    a=x;  
    b=y;  
}
```

- Los atributos **ref** y **fijo** hay que inicializarlos obligatoriamente mediante inicializadores: no puede asignarse en el cuerpo de la función ni inicializarlas al declararlas.

```
class cla {  
    int a,b, &ref=a; //ERROR  
    const float fijo=9.8; //ERROR  
public:  
    cla();  
    cla(int x, int y, float v);  
    void set(int x, int y): a(x), b(y) { }  
}; //ERROR set no es un constructor
```

```
class cla {  
    int a,b, &ref;  
    const float fijo;  
public:  
    cla() { a=0; b=0; ref=a; fijo=9.8; } //ERROR  
    cla(int x, int y, float v) {  
        a=x; b=y; ref=a; fijo=v; } //ERROR  
};
```

6. Constructores y Destructores

6.3 Constructor por defecto

- **Constructor por defecto es aquel que no tiene parámetros** o que, si los tiene, todos sus argumentos tienen asignado un valor por defecto, **es decir, aquel constructor que puede ser llamado sin tener que pasarle ningún argumento.**
- **Es necesario si se quiere poder crear un objeto de la forma:**

```
class objeto; //se invoca el constructor sin argumentos
```

y también cuando se quiere crear un array de objetos:

```
class array[10]; //se invoca el constructor sin argumentos para
                //cada objeto del array
```

ya que no es posible pasar argumentos propios para cada uno de los objetos del array para su inicialización (debe haber un constructor sin argumentos, que es el que se invoca)

6.4 Constructor de oficio

- **Si el programador no define ningún constructor, el compilador crea uno de oficio**

```
class::class() { //constructor de oficio, sin argumentos
                //no hace nada (lo crea el compilador si no definimos ninguno)
}
```

- **Si el programador define algún constructor para la clase (con o sin argumentos), el compilador no crea el constructor de oficio.**

Si el usuario define una clase con un/os constructor/es con argumentos y ningún constructor por defecto entonces no podrá crear un array de objetos (ni un objeto sin indicar el valor de sus atributos).

Ejemplo:

```
#include <iostream>
using namespace std;

class hour {
    int hora, mint;
public:
    hour(int h, int m):hora(h+m/60),mint(m%60){ }
    hour(int m):hora(m/60), mint(m%60){ }
    hour():hora(0) { mint=0; }
    friend ostream& operator<<(ostream &s,
                              const hour &t);
};

ostream& operator<<(ostream &s, const hour &t) {
    s << t.hora << ":" << t.mint;
    return s; //s actúa a modo de cout
}

int main() {
    hour a(2,7); //hour a=hour(2,7);
    hour b(90); //hour b=hour(90);
    hour c;     //hour c=hour();
    hour t[3];  //hour t[3]={hour(),hour(),hour()};
    for(int i=0,j=30; i<3; i++,j+=20)
        t[i]=hour(i+1,j);
    cout << a << "," << b << "," << c << endl;
    cout << t[0] << "," << t[1] << "," << t[2];
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Pantalla:

```
2:7 , 1:30 , 0:0
1:30 , 2:50 , 4:10
```

Si no tuviera el constructor por defecto

```
class hour {
    int hora, mint;
public:
    hour(int h, int m);
    hour(int m);
    hour(); //constructor por defecto
};
```

El compilador no lo genera, al tener la clase otros constructores, por lo que:

```
int main() {
    hour a(2,7), b(90);
    hour c;      //ERROR
    hour t[3];   //ERROR
    ...
}
```

Para que no diera error habría que poner:

```
int main() {
    hour a(2,7), b(90);
    hour c(0);
    hour t[3]={hour(0),hour(0),hour(0)};
    ...
}
```

6.5 Constructor de copia

- Cuando se crea un objeto inicializándolo a partir de otro objeto de la misma clase se llama a un constructor especial llamado **constructor de copia (copy constructor)**.

- Sintaxis:

```
clase::clase(const clase &objeto); //constructor de copia
```

- Si el programador no define un constructor de copia (aunque haya definido otros constructores con o sin argumentos), el compilador crea un constructor de copia de oficio, el cual realiza una copia bit a bit de las variables miembro del objeto original pasado como parámetro, al objeto creado que lo invoca.

```
clase::clase(const clase &objeto) {  
    sentencias_que_hacen_copia_binaria; //constructor de copia  
}
```

- Si el comportamiento del constructor de copia de oficio no es el deseado el programador deberá definir uno propio.
- Cuando no hay punteros implicados, el constructor de copia de oficio funciona bien. Cuando hay atributos punteros, el constructor de copia de oficio no funciona bien ya que al copiar binariamente hace que tanto el atributo puntero del objeto copiador como del objeto copia apunten al mismo sitio.
- El constructor de copia se invoca en los siguientes casos:
 - ☐ cuando se declara un objeto inicializándolo a partir de otro objeto de la misma clase.
 - ☐ cuando a un método o una función se le pasan objetos por valor.
 - ☐ cuando un método o una función devuelve un objeto como valor de retorno.

En esos casos hay que crear copias del objeto y para ello se usa el constructor de copia.

Ejemplo 1:

```
class x, y; //constructor por defecto (se crea los objetos x y)  
class a = x; //constructor de copia (se crea el objeto a)  
class b(x); //constructor de copia (se crea el objeto b)  
y = x; //operador de asignación (se modifica y)
```

En la 2ª sentencia se crea un objeto **a** inicializando sus variables miembros con los mismos valores que tienen en **x**. La 3ª sentencia es una forma sintáctica equivalente a la 2ª: **b** se inicializa con los valores de **x**. La 4ª sentencia es una asignación (no se crea un nuevo objeto sino que se trabaja con objetos ya creados)

Las sentencias 2ª y 3ª anteriores se ejecutan aunque no se haya definido en la clase ningún constructor de copia, ya que en ese caso, el compilador crea uno de oficio.

Ejemplo 2:

```
class clase {  
    ...  
public:  
    clase proc(clase a, clase &b);  
};  
clase clase::proc(clase a, clase &b) {  
    clase aux; //se invoca constructor por defecto  
    ...  
    return aux; //se invoca constructor de copia  
} //para devolver una copia de aux  
int main() {  
    clase x,y,c,z;  
    c = x.proc(y,z); //invoca constructor copia  
} //ya que y se pasa por valor
```

En el **main()** al invocar el objeto **x** el método **proc()** pasa el objeto **y** por valor y **z** por referencia, por lo que para copiar el objeto **y** en el parámetro **a** del método se invoca el constructor de copia.

Al terminar el método **proc()** debe devolver una copia del objeto local **aux**, por lo que de nuevo se invoca el constructor de copia para asignar al objeto **c** del **main()** una copia de **aux**.

6.6 Necesidad de escribir un constructor de copia y un operador =

- Como norma general, cuando una clase trabaje con memoria dinámica (por tener un atributo/variable miembro tipo puntero), se debe definir los siguientes métodos:
 - **Un constructor** (que reserve memoria e inicialice el resto de variables miembros).
Si pretendemos crear arrays de objetos la clase debe tener un **constructor por defecto**.
 - **Un constructor de copia** (que evite que al pasar un objeto por copia o cree un objeto a partir de otro, sus variables miembros tipo puntero apunten a la misma zona de memoria).
 - **Un destructor** (que libere la memoria dinámica).
 - **Un operador de asignación = sobrecargado** (que evite que la variable miembro tipo puntero del objeto asignado apunte al mismo lugar que la del objeto del que se copia).
- Como los constructores, destructores y operador de asignación = sobrecargado no se heredan, una clase derivada de una clase base debe también implementar estos métodos.

Ejemplo: Elimine solo operator=. Elimine solo constructor copia. Elimine ambos. Vea lo que ocurre

```
#include <iostream> //cin, cout
using namespace std;

class cad {
    char* s;
    int n;
public:
    cad() { s=new char[1]; strcpy(s,""); n=0; }
    cad(const char*); // c. general
    cad(const cad&); // c. de copia
    ~cad() { delete [] s; }
    void setcad(const char*);
    cad& operator=(const cad &);
    cad operator+(const cad &) const;
    bool operator==(cad) const;
    friend ostream& operator<<(ostream&, const cad&);
    void ver() const { cout << n << ", " << s; }
};

cad::cad(const char* c) {
    n = strlen(c);
    s = new char[n + 1];
    strcpy(s, c);
}

cad::cad(const cad& cd) {
    n = cd.n;
    s = new char[n+1];
    strcpy(s, cd.s);
}

void cad::setcad(const char* c) {
    n = strlen(c);
    delete [] s;
    s = new char[n+1];
    strcpy(s, c);
}

cad& cad::operator=(const cad &cd) {
    if (this != &cd) {
        n = cd.n;
        delete [] s;
        s=new char[n+1];
        strcpy(s, cd.s);
    }
    return *this;
}
```

Pantalla:

```
6,Blanca 8,Avestruz 6,Paloma
8,Avestruz 8,Avestruz
Avestruz == Avestruz
Gris,Blanca,Blanca
T[0] obj 2686672: Gris
T[1] obj 2686680: GrisRojo
T[2] obj 2686688: GrisGris
```

```
cad cad::operator+(const cad& b) const {
    cad c;
    c.n = n + b.n;
    c.s = new char[c.n + 1];
    strcpy(c.s, s);
    strcat(c.s, b.s);
    return c;
}

bool cad::operator==(cad b) const {
    if (n != b.n) return false;
    return (strcmp(s,b.s)==0);
}

ostream& operator<<(ostream& o, const cad& c) {
    o << c.s;
    return o;
}

int main() {
    cad c1; c1 = "Blanca";
    cad c2("Paloma");
    cad c3 = c2;
    c2.setcad("Avestruz");
    c1.ver(); c2.ver(); c3.ver(); cout<<endl;
    cad c4(c2);
    c2.ver(); c4.ver(); cout << endl;
    if (c2 == c4)
        cout << c2 << " == " << c4 << endl;
    (c3 = c2) = c1; //c3=c2; c3=c1;
    c3 = c2 = c1; //c2=c1; c3=c2;
    c1.setcad("Gris");
    cout << c1 << ", " << c2 << ", " << c3 << endl;
    c2 = c2;
    c2 = c1;
    c2 = c1 + "Rojo";
    c3 = c1 + c1;
    cad T[3];
    T[0]=c1; T[1]=c2; T[2]=c3;
    for(int i=0; i<3; i++)
        cout << "T[" << i << "] objeto "
            << (long)&T[i] << ":" << T[i] << endl;
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Para entender lo que ocurre vamos a añadir mensajes a los métodos:

<pre>#include <iostream> //cin, cout using namespace std; class cad { int n; char* s; public: cad(); //constructor por defecto cad(const char*); //constructor general cad(const cad&); //constructor de copia ~cad(); //destructor void setcad(const char*); cad& operator=(const cad &); cad operator+(const cad &) const; bool operator==(cad) const; friend ostream& operator<<(ostream&, const cad&); void ver() const; }; cad::cad() { s = new char[1]; strcpy(s, ""); n = 0; cout << " Constructor por defecto: crea " << (long)this << " " << (long)s << endl; } cad::cad(const char* c) { n=strlen(c); s=new char[n+1]; strcpy(s,c); cout << " C. general: crea " << (long)this << " " << (long)s << " desde " << c <<"\n"; } cad::cad(const cad& cd) { n=cd.n; s=new char[n+1]; strcpy(s,cd.s); cout << " C. copia: crea " << (long)this <<" " <<(long)s<<" desde " <<(long)&cd<< endl; } cad::~~cad() { // destructor delete [] s; cout << " Destructor de " << (long)this << endl; } void cad::setcad(const char* c) { n = strlen(c); delete [] s; s = new char[n + 1]; strcpy(s, c); cout << " setcad: " << (long)this << " " << (long)s << endl; } cad& cad::operator=(const cad &cd) { if(this != &cd) { n = cd.n; delete [] s; s=new char[n+1]; strcpy(s, cd.s); cout << " Operador asignacion: " << (long)this << " = " << (long)&cd << endl; } return *this; }</pre>	<pre>cad cad::operator+(const cad& b) const { cout << " "; cout << (long)this << ".operator+(" << (long)&b << ")\n"; cad c; c.n = n + b.n; c.s = new char[c.n + 1]; strcpy(c.s, s); strcat(c.s, b.s); return c; } bool cad::operator==(cad b) const { cout << " "; cout << (long)this << ".operator==(" << (long)&b << ")\n"; if (n != b.n) return false; return (strcmp(s,b.s)==0); } ostream& operator<<(ostream& o, const cad& c) { o << c.s; return o; } void cad::ver() const { cout << (long)this << " n= " << n << " s=" << (long)s << s << endl; } int main() { { //no es un error, es intencionado cad c1; c1 = "Blanca"; cad c2("Paloma"); cad c3 = c2; c2.setcad("Avestruz"); c1.ver(); c2.ver(); c3.ver(); cad c4(c2); c2.ver(); c4.ver(); if (c2 == c4) cout << c2 << " == " << c4 << endl; (c3 = c2) = c1; //c3=c2; c3=c1; c3 = c2 = c1; //c2=c1; c3=c2; c1.setcad("Gris"); cout << c1 << ", " << c2 << ", " << c3 << endl; c2 = c2; c2 = c1; c2 = c1 + "Rojo"; c3 = c1 + c1; cad T[3]; T[0]=c1; T[1]=c2; T[2]=c3; for(int i=0; i<3; i++) cout << "T[" << i << "]" objeto " << (long)&T[i] << ":" << T[i] << endl; } //para ver mensajes del destructor system("PAUSE"); return EXIT_SUCCESS; }</pre>
---	---

Elimina la sobrecarga del operador de asignación = y vea lo que ocurre.

Elimina el constructor copia y vea lo que ocurre.

Elimina ambos (operador de asignación y constructor copia) y vea lo que ocurre.

En la sobrecarga del operador + pasa el parámetro por valor en vez de por referencia constante y vea lo que ocurre: `cad cad::operator+(cad b) const;`

Ejercicio: Ejecuta el siguiente programa e intenta comprender a qué son debidos los errores que se producen al ejecutarlo. Corrígelos

Sobrecarga de <<

```
Punto
- static int numserie
- int *identificador
- int x
- int y
+ Punto( int x, int y )
+ ~Punto()
+ int operator==( Punto p )
+ Punto operator=( Punto p )
Friend: ostream& operator<<
( ostream &salida,
const Punto p )
```

En funciones friend:
Primer parámetro: operando de izquierda.
Segundo parámetro: operando de la derecha (en binarios).
En miembros, a la izquierda va el objeto.
Se usan friend para conmutatividad también.

Cuando una función friend se define fuera de la clase, no se escribe friend ni el resolutor de ámbito. La función no es realmente un método.

Existen casos especiales con el operador de asignación cuando el objeto que se copia se declaró como un puntero. Hay que comprobar si se copia el mismo o si se le asigna un NULL.

Pantalla:

```
0:(10,15) -- 1:(10,15) -- 2:(0,0)
0:(10,15) -- 5383304:(10,15) -- 5383408:(0,0)
5383424:(10,15) == 5375160:(10,15)
5374344:(10,15) != 5375160:(10,16)
5374344:(10,15) -- 5375160:(10,15) -- 5383408:(10,15)
La ultima línea no se ejecuta... el programa aborta...
```

```
#include <cstdlib>
#include <iostream>

using namespace std;

class Punto {
    static int numserie;
    int *identificador;
    int x,y;
public:
    Punto(int nx, int ny) { x=nx; y=ny; identificador=new int(numserie++); }
    ~Punto() { delete identificador; }
    int getx() { return x; }
    int gety() { return y; }
    void setxy(int x, int y) { this->x=x; Punto::y=y; }
    int operator==( Punto p ) { return (x==p.x)&&(y==p.y); }
    Punto operator=( Punto p ) {
        x=p.x, y=p.y; /*no se sobrescribe la serie*/
        return p;
    }
    friend ostream& operator<<( ostream &salida, const Punto p );
};

int Punto::numserie=0;

ostream& operator<<( ostream &salida, const Punto p ) {
    // acceso a datos protected y private...
    salida << *(p.identificador) << ":" << p.x << ", " << p.y << "";
    return salida;
}

Punto::operator char*() { //en vez de operator<<
char salida[30];
sprintf( salida, "%i:(%i,%i)",*identificador, x, y );
return strdup( salida );
}

int main(int argc, char *argv[]) {
    Punto p1(10,15), p2(10,15), p3(0,0);
    cout << p1 << " -- " << p2 << " -- " << p3 << endl;
    cout << p1 << " -- " << p2 << " -- " << p3 << endl;
    cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ;
    p2.setxy(p2.getx(), p2.gety()+1);
    cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ;
    p3=p2=p1=p1;
    cout << p1 << " -- " << p2 << " -- " << p3 << endl;
    cout << p1 << ((p1==p3)? " == ":" != ") << p3 << "\n" ;

    system("PAUSE");
    return EXIT_SUCCESS;
}
```

Operadores sobrecargables:

+ - * / % ^ & | ~ ! = < > += -= *= /= %= ^= &= |= << >> >>= <<= == != <= >= && || ++ -- ->* , -> [] () new new[] delete delete[]

Operadores no sobrecargables:

. .* :: ?: sizeof

Con la sobrecarga del operador << es necesario utilizar la funcion friend. Es posible tener comportamiento similar **sobrecargando char*** en vez de << (no recomendado: memoria no se libera)

Solucion:

Los errores son debidos a que hay memoria dinámica (el atributo **identificador** es de tipo puntero) y no se ha definido un constructor de copia, por lo que el que genera de oficio el compilador hace una copia binaria de los datos → cuando se pasa por valor la copia apunta a la misma zona de memoria y al destruirse la copia y ejecutarse el destructor se libera la memoria del original.

En la sobrecarga de los operadores ==, = y << se pasa por copia, por lo que al ejecutar cualquiera de dichos operadores se producen errores.

Sol1: definir constructor de copia (recomendable)	Sol2: pasar todo por referencia
<pre>#include <cstdlib> #include <iostream> using namespace std; class Punto { static int numserie; int *identificador; int x,y; public: Punto(int nx, int ny) { x=nx; y=ny; identificador=new int(numserie++); } Punto(const Punto &p) { x=p.x; y=p.y; identificador=new int(*(p.identificador)); } ~Punto() { delete identificador; } int getx() { return x; } int gety() { return y; } void setxy(int x, int y) { this->x=x; Punto::y=y; } int operator==(Punto p) { return (x==p.x)&&(y==p.y); } Punto operator=(Punto p) { x=p.x, y=p.y; /*no se sobrescribe la serie*/ return p; } friend ostream& operator<<(ostream &s, const Punto p); }; int Punto::numserie=0; ostream& operator<<(ostream &s, const Punto p) { // acceso a datos protected y private... s << *(p.identificador) << ":" << p.x << ", " << p.y << " "; return s; } int main(int argc, char *argv[]) { Punto p1(10,15), p2(10,15), p3(0,0); cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p2.setxy(p2.getx(), p2.gety()+1); cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p3=p2=p1=p1; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p3)? " == ":" != ") << p3 << "\n" ; system("PAUSE"); return EXIT_SUCCESS; }</pre>	<pre>#include <cstdlib> #include <iostream> using namespace std; class Punto { static int numserie; int *identificador; int x,y; public: Punto(int nx, int ny) { x=nx; y=ny; identificador=new int(numserie++); } ~Punto() { delete identificador; } int getx() { return x; } int gety() { return y; } void setxy(int x, int y) { this->x=x; Punto::y=y; } int operator==(Punto &p) { return (x==p.x)&&(y==p.y); } Punto& operator=(Punto &p) { x=p.x, y=p.y; /*no se sobrescribe la serie*/ return p; } friend ostream& operator<<(ostream &s, const Punto &p); }; int Punto::numserie=0; ostream& operator<<(ostream &s, const Punto &p) { // acceso a datos protected y private... s << *(p.identificador) << ":" << p.x << ", " << p.y << " "; return s; } int main(int argc, char *argv[]) { Punto p1(10,15), p2(10,15), p3(0,0); cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p2.setxy(p2.getx(), p2.gety()+1); cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p3=p2=p1=p1; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p3)? " == ":" != ") << p3 << "\n" ; system("PAUSE"); return EXIT_SUCCESS; }</pre>

Pantalla:

```
0:(10,15) -- 1:(10,15) -- 2:(0,0)
0:(10,15) -- 1:(10,15) -- 2:(0,0)
0:(10,15) == 1:(10,15)
0:(10,15) != 1:(10,16)
0:(10,15) -- 1:(10,15) -- 2:(10,15)
0:(10,15) == 2:(10,15)
```

Presione una tecla para continuar . . .

Solucion:

Los errores son debidos a que hay memoria dinámica (la variable miembro **identificador** es de tipo puntero) y no se ha definido un constructor de copia, por lo que el que genera de oficio el compilador hace una copia binaria de los datos → cuando se pasa por valor la copia apunta a la misma zona de memoria y al destruirse la copia y ejecutarse el destructor se libera la memoria del original.

En la sobrecarga de los operadores ==, = y << se pasa por copia, por lo que al ejecutar cualquiera de dichos operadores se producen errores.

Sol1: definir constructor de copia (recomendable)	Sol: constructor de copia y paso por referencia const (mejor)
<pre>#include <cstdlib> #include <iostream> using namespace std; class Punto { static int numserie; int *identificador; int x,y; public: Punto(int nx, int ny) { x=nx; y=ny; identificador=new int(numserie++); } Punto(const Punto &p) { x=p.x; y=p.y; identificador=new int(*(p.identificador)); } ~Punto() { delete identificador; } int getx() { return x; } int gety() { return y; } void setxy(int x, int y) { this->x=x; Punto::y=y; } int operator==(Punto p) { return (x==p.x)&&(y==p.y); } Punto operator=(Punto p) { x=p.x, y=p.y; /*no se sobrescribe la serie*/ return p; } friend ostream& operator<<(ostream &s, const Punto p); }; int Punto::numserie=0; ostream& operator<<(ostream &s, const Punto p) { // acceso a datos protected y private... s << *(p.identificador) << ":" << p.x << " , " << p.y << " "; return s; } int main(int argc, char *argv[]) { Punto p1(10,15), p2(10,15), p3(0,0); cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p2.setxy(p2.getx(), p2.gety()+1); cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p3=p2=p1=p1; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p3)? " == ":" != ") << p3 << "\n" ; system("PAUSE"); return EXIT_SUCCESS; }</pre>	<pre>#include <cstdlib> #include <iostream> using namespace std; class Punto { static int numserie; int *identificador; int x,y; public: Punto(int nx, int ny) { x=nx; y=ny; identificador=new int(numserie++); } Punto(const Punto &p) { x=p.x; y=p.y; identificador=new int(*(p.identificador)); } ~Punto() { delete identificador; } int getx() { return x; } int gety() { return y; } void setxy(int x, int y) { this->x=x; Punto::y=y; } int operator==(const Punto &p) { return (x==p.x)&&(y==p.y); } Punto& operator=(const Punto &p) { if (this != &p) { //por si es p=p x=p.x, y=p.y; /*no se sobrescribe la serie*/ } return *this; } friend ostream& operator<<(ostream &s, const Punto &p); }; int Punto::numserie=0; ostream& operator<<(ostream &s, const Punto &p) { // acceso a datos protected y private... s << *(p.identificador) << ":" << p.x << " , " << p.y << " "; return s; } int main(int argc, char *argv[]) { Punto p1(10,15), p2(10,15), p3(0,0); cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p2.setxy(p2.getx(), p2.gety()+1); cout << p1 << ((p1==p2)? " == ":" != ") << p2 << "\n" ; p3=p2=p1=p1; cout << p1 << " -- " << p2 << " -- " << p3 << endl; cout << p1 << ((p1==p3)? " == ":" != ") << p3 << "\n" ; system("PAUSE"); return EXIT_SUCCESS; }</pre>

Pantalla:

0:(10,15) -- 1:(10,15) -- 2:(0,0)

0:(10,15) -- 1:(10,15) -- 2:(0,0)

0:(10,15) == 1:(10,15)

0:(10,15) != 1:(10,16)

0:(10,15) -- 1:(10,15) -- 2:(10,15)

0:(10,15) == 2:(10,15)

Presione una tecla para continuar . . .

Reto. Dada la siguiente clase:

```
#include <cstdlib>
#include <iostream>

using namespace std;

class Punto {
    int x,y;
public:
    Punto() { x = 0; y = 0; }
    Punto(int a) { x = a; y = 0; }
    Punto(int a, int b) { x = a; y = b; }
    Punto add(Punto p) {
        x+=p.x; y+=p.y; return (*this);
    }
    Punto add(Punto p, Punto q) {
        x=x+p.x+q.x;
        y+=p.y+q.y;
        return (*this);
    }
    void inc23() { x+=2; y+=3; }
    void inc23(Punto p) { x+=p.x; y+=p.y; }
    void ver() const { cout << x <<" "<< y << endl; }
};

int main() {
    Punto x(3,2), y;
    const Punto z=1; //const Punto z(1); const Punto z=Punto(1);
    cout << "x, y, z valen:\n";
    x.ver(); y.ver(); z.ver();
    y.add(x, z);
    x=y.add(z);
    cout << "x, y, z valen:" << endl;
    x.ver(); y.ver(); z.ver();
    x.inc23(); y.inc23(z);
    cout << "x, y, z valen:" << endl;
    x.ver(); y.ver(); z.ver();

    //haz que sea posible ejecutar las lineas comentadas sin cambiar nada en la clase Punto
    //excepto añadir un método a la clase Punto y una funcion amiga
    /*
    x = y + z; cout << "x: "; x.ver();
    x = z + x; cout << "x: "; x.ver();
    y = z + 4; cout << "y: "; y.ver();
    y = 2 + z + y +1 ; cout << "y: "; y.ver();
    cout << "x: " << x << " y: " << y << " z: " << z << endl;
    */
    system("PAUSE"); return EXIT_SUCCESS;
}
```

SALIDA POR PANTALLA

```
x, y, z valen:
3,2
0,0
1,0
x, y, z valen:
5,2
5,2
1,0
x, y, z valen:
7,5
6,2
1,0
Presione una tecla para continuar . . .
```

Haz que sea posible ejecutar las líneas comentadas sin cambiar absolutamente nada en la clase Punto excepto añadir un nuevo método y una función amiga a la clase Punto.

7. Miembros Estáticos (atributos y métodos)

- Cada objeto tiene su propia copia de cada variable miembro de la clase. Dichos miembros no existen hasta que no creamos un objeto.
- C++ permite definir miembros comunes a todos los objetos (sólo hay una copia para todos los objetos en lugar de una por cada objeto), **declarando dicho miembro estático**.
Ej: sueldo base común a todos los empleados, interés bancario común a todas las cuentas de un tipo, contador de objetos.
- Los atributos y métodos **static** resultan útiles para establecer variables y métodos **comunes** a todos los objetos de una clase
- Para acceder a los miembros estáticos (**public** o **private**) se usa la siguiente notación:

`clase::miembro_estatico` (recomendable, posible siempre, exista o no objetos clase)

`objeto.miembro_estatico` (más confuso, sólo posible cuando ya existe objeto)

Se recomienda la 1ª forma ya que la 2ª es válida también para atributos no estáticos.
(véase `main()` en pág 44)

7.1 Atributos estáticos (static):

- Son compartidos por todos los objetos de una clase. No aumentan el tamaño de la clase ya que su almacenamiento es global.
- Se utilizan para almacenar información común (compartida) a todos los objetos de la clase
- Existen aunque no se haya creado ningún objeto de dicha clase (atributo `cuenta` en `main()` pág 44)
- **Preferibles a las variables globales** ya que conservan la encapsulación: son como variables globales, pero con ámbito y acceso restringido a nivel de clase (**public**, **protected** o **private**).
- Pueden ser tomados como valores por defecto en argumentos de métodos (estáticos o no)
- No se pueden inicializar en un constructor (porque se inicializarían muchas veces, cada vez que creáramos un objeto). Necesitan ser definidos e inicializados fuera de la clase:
`tipo clase::atributo_estatico = valor_inicial;` (int `Objeto::cuenta=0;` ejemplo pág 44)
- Pueden ser un objeto de la clase que los contiene. (la clase `Punto` tiene un atributo `static Punto` o en pág 45)

7.2 Métodos estáticos (static):

- Se usan generalmente para actuar globalmente sobre todos los objetos de una clase.
- No tienen el puntero **this** que tienen los métodos normales, por lo que sólo pueden acceder y llamar a los atributos y métodos **static** que haya en la clase (aunque puede llamar a cualquier miembro de los objetos pasados por parámetro). (método `Objeto::verCuenta()` en pág 44)
- No pueden ser métodos **const** (ya que no actúan sobre ningún objeto concreto de la clase)
- Pueden recibir como argumentos objetos de su propia clase. (véase método `Punto::distancia()` en pág 45)
- Son una alternativa a las funciones **friend** para conseguir simetría (cuando se desee que un método actúe sobre 2 objetos de una misma clase) (función `distancia()` vs método `Punto::distancia()` en pág 45)

Static: atributos y métodos de clase:

Tenemos una clase con una variable común a toda la clase: cuenta de instancias.

Métodos **static** sólo pueden acceder a las variables y métodos **static** de su clase

Objeto
+ static int cuenta = 0
- const char *nombre
+ Objeto(const char *nuevoNombre = "")
+ ~Objeto()
+ static void verCuenta()
+ void ver() const

Ojo: static se pone en el .hpp pero no en el .cpp

Inicialización de variables de clase static (no en el constructor sino fuera de la clase).

Objeto *array = new Objeto[5]; crea array dinámico de 5 elementos tipo Objeto.

Para poder crear arrays de objetos es necesario que la clase tenga un constructor por defecto (sin argumentos o que pueda invocarse sin argumentos).

Si observamos el constructor puede invocarse sin argumentos ya que el único parámetro que tiene es opcional **Objeto(const char *nuevoNombre = "")**;

```
#include <iostream>
using namespace std;

class Objeto {
    const char * nombre;
public:
    static int cuenta;
    static const float pi;
    //constructor con parámetro por defecto igual a ""
    Objeto( const char *nuevoNombre = "" ) {
        nombre = strdup(nuevoNombre);
        cuenta++;
    }
    virtual ~Objeto() { delete [ ] nombre; cuenta--; }
    static void verCuenta() {
        //cout << nombre; No permitido
        cout << cuenta;
    }
    void ver() const { cout << cuenta << ":" << nombre << "\n"; }
};

int Objeto::cuenta=0;
const float Objeto::pi=3.14;

int main(int argc, char *argv[]) {
    cout << Objeto::cuenta << "\n"; //recomendable
    Objeto *array = new Objeto[5]; //crea 5 objetos
    Objeto obj("Pieza");
    cout << array[3].cuenta << endl; //confuso
    obj.verCuenta(); //confuso
    cout << "\n";
    delete [ ] array; //probar con "delete array;"...
    cout << obj.cuenta << "\n"; //confuso
    Objeto::verCuenta(); //recomendable
    cout << "\n";
    obj.ver();
    system("PAUSE"); return EXIT_SUCCESS;
}
```

No es aconsejable definir atributos estáticos públicos, es mejor definirlos privados y proporcionar un método estático público para acceder a ellos y modificarlos (en el método estático podremos controlar la forma en la que permitimos hacerlo)

```
class Objeto {
    const char * nombre;
    static int cuenta;
    static const float pi;
public:
    Objeto( const char *nuevoNombre = "" ) {
        nombre = strdup(nuevoNombre);
        cuenta++;
    }
    virtual ~Objeto() { delete [ ] nombre; cuenta--; }
    static void verCuenta() { cout << cuenta; }
    void ver() const { cout << cuenta << ":" << nombre << "\n"; }
    static int getcuenta() { return cuenta; }
    static int setcuenta(int i) { cuenta=i; }
    static const float getpi() { return pi; }
};
```

```
int Objeto::cuenta=0;
const float Objeto::pi=3.14;

int main(int argc, char *argv[]) {
    cout << Objeto::getcuenta() << "\n";
    Objeto *array = new Objeto[5]; //crea 5 objetos
    Objeto obj("Pieza");
    cout << array[3].getcuenta() << "\n";
    obj.muestraCuenta();
    cout << "\n";
    delete [ ] array; //probar con "delete array;"...
    cout << obj.getcuenta() << "\n";
    Objeto::verCuenta();
    cout << "\n";
    obj.ver();
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Que los atributos estáticos puedan ser objetos de la clase que los contiene no significa que podemos tener datos recursivos, sino que podemos definir un objeto común para todos los objetos que instancien la clase

```
#include <iostream>
#include <cmath>
using namespace std;

class Punto {
    int x, y;
public:
    static Punto o; //origen comun para todos los puntos
    static int n;
    Punto(int a=0, int b=0) { x = a; y = b; n++; }
    void set(int a, int b) { x = a; y = b; }
    friend float distancia(const Punto &, const Punto &);
    static float distancia(const Punto &, const Punto &);
    void dibujar() const {
        cout << "(" << x+o.x << ", " << y+o.y << ") \n"; //muestra el punto tomando o como origen de coordenadas
    }
};

Punto Punto::o; // Punto Punto::o(0,0); //forma de definir e inicializar los miembros staticos
int Punto::n = 0; //fuera de la clase! Estas sentencias son obligatorias!!!

float distancia(const Punto &a, const Punto &b) {
    return sqrt(pow((b.x-a.x),2.0)+pow((b.y-a.y),2.0));
}

float Punto::distancia(const Punto &a, const Punto &b) {
    return sqrt(pow((b.x-a.x),2.0)+pow((b.y-a.y),2.0));
}

int main(int argc, char *argv[]) {
    cout << "Hay " << Punto::n << " puntos creados\n";
    Punto::o.dibujar();
    Punto::n--; //para que no cuente el punto origen
    Punto p1(1,1), p2(5,4);
    cout << "Hay " << Punto::n << " puntos creados\n";
    p1.dibujar();
    p2.dibujar();
    cout << distancia(p1,p2) << endl;
    //cambio el origen de coordenadas a 2,3
    p1.o.set(2,3); //confuso //Punto::o.set(2,3); //recomendado p2.o.set(2,3); //confuso
    p1.dibujar();
    p2.dibujar();
    cout << Punto::distancia(p1,p2) << endl; //recomendado //la distancia no cambia
    //cout << p1.distancia(p1,p2) << endl; //confuso
    //Punto::distancia(p1,p2) es lo mismo que p2.distancia(p1,p2) y que p1.distancia(p1,p2)
    //cambio el origen de coordenadas al indicado en p2
    p1.o=p2; //confuso //Punto::o=p2; //recomendado p2.o=p2; //confuso
    p1.dibujar();
    p2.dibujar();
    cout << distancia(p1,p2) << endl; //la distancia no cambia
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Pantalla:

```
Hay 1 puntos creados
(0,0)
Hay 2 puntos creados
(1,1)
(5,4)
5
(3,4)
(7,7)
5
(6,5)
(10,8)
5
```

Como podemos observar, la distancia la podemos calcular mediante una función miembro estática o mediante una función no miembro (amiga).

La forma en la que se llama en el main() difiere una de otra. Para invocar la función miembro estática hay que utilizar la notación `clase::funcion()` **recomendo** o `objeto.funcion()` **confuso** mientras que para invocar la función amiga simplemente utilizamos la notación `funcion()`.

No es buena técnica definir atributos estáticos públicos, es mejor definirlos privados y proporcionar un método estático público para acceder a ellos y modificarlos (en el método estático podremos controlar la forma en la que permitimos hacerlo).

También es conveniente utilizar la notación `clase::funcion()` y la notación `clase::atributo` para hacer referencia a miembros estáticos, ya que esta notación es exclusiva de miembros estáticos.

```
#include <iostream>
#include <cmath>
using namespace std;
class Punto {
    int x, y;
    static Punto o; //origen comun para todos los puntos
    static int n;    //los declaro privados
public:
    Punto(int a=0, int b=0) { x = a; y = b; n++; }
    void set(int a, int b) { x = a; y = b; }
    friend float distancia(const Punto &, const Punto &);
    static float distancia(const Punto &, const Punto &);
    static int getn() { return n; }
    static void setn(int n) { Punto::n=n; } //para distinguir n de Punto::n
    static Punto getorigen() { return o; }
    static void setorigen(Punto p) { o = p; }
    void dibujar() const {
        cout << "(" << x+o.x << ", " << y+o.y << ") \n"; //muestra el punto tomando o como origen de coordenadas
    }
};

Punto Punto::o; // Punto Punto::o(0,0); //forma de definir e inicializar los
int Punto::n = 0; // miembros estáticos fuera de la clase! Estas sentencias son obligatorias!!!

float distancia(const Punto &a, const Punto &b) {
    return sqrt(pow((b.x-a.x),2.0)+pow((b.y-a.y),2.0));
}

float Punto::distancia(const Punto &a, const Punto &b) {
    return sqrt(pow((b.x-a.x),2.0)+pow((b.y-a.y),2.0));
}

int main(int argc, char *argv[]) {
    cout << "Hay " << Punto::getn() << " puntos creados\n";
    Punto::getorigen().dibujar(); Punto::getorigen().set(7,7); //modifica una copia
    Punto::setn(Punto::getn()-1); //para que no cuente el punto origen
    Punto p1(1,1), p2(5,4);
    cout << "Hay " << Punto::getn() << " puntos creados\n";
    p1.dibujar();
    p2.dibujar();
    cout << distancia(p1,p2) << endl;
    //cambio el origen de coordenadas a 2,3
    Punto::setorigen(Punto(2,3)); //recomendado //p2.setorigen(Punto(2,3)); //confuso
    p1.dibujar();
    p2.dibujar();
    cout << Punto::distancia(p1,p2) << endl; //recomendado //la distancia no cambia
    //cout << p1.distancia(p1,p2) << endl; //confuso
    //cambio el origen de coordenadas al indicado en p2
    Punto::setorigen(p2); //recomendado //p1.setorigen(p2); //confuso
    p1.dibujar();
    p2.dibujar();
    cout << distancia(p1,p2) << endl; //la distancia no cambia
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Pantalla:

Hay 1 puntos creados
(0,0)
Hay 2 puntos creados
(1,1)
(5,4)
5
(3,4)
(7,7)
5
(6,5)
(10,8)
5

8. Consideraciones a tener en cuenta

8.1 Como leer una cadena compuesta (frase) desde teclado

cin ignora los espacios y saltos de línea (separadores) que encuentra antes de la secuencia que pretende leer. Los caracteres separadores que haya después, y esto es **importante**, incluidos los espacios y el salto de línea, se mantienen en el buffer de entrada.

Esto permite a **cin** leer números y cadenas desde teclado, pero no permite leer frases (cadenas de caracteres con espacios) desde teclado ya que tras leer la primera palabra de una frase deja de leer del buffer del teclado al encontrar espacios o saltos de línea.

Si necesitamos leer una frase o un nombre compuesto desde teclado debemos usar otras funciones:

-gets(char *): permite leer una cadena desde teclado hasta encontrar un salto de línea

-cin.getline(char *, int size, char delim='\n'): permite leer una cadena de hasta **size-1** caracteres de longitud desde teclado (**cin**) o hasta encontrar el carácter indicado en **delim** (por defecto salto de línea).

```
cin.getline(cad, 20); //cin.getline(cad, 20, '\n');
```

Es decir, ambas funciones leen lo que haya en el buffer del teclado hasta encontrar un salto de línea o el carácter delimitador **delim** indicado en **getline()**.

Problema:

-cin deja en el buffer del teclado los caracteres separadores (espacios y salto de línea) que hay después de la secuencia que pretende leer.

-un gets o getline que haya tras un **cin** leerá los separadores y salto de línea que el **cin** ha dejado en el buffer del teclado y no se comportará de forma correcta.

Para evitar esto comportamiento erróneo debemos asegurarnos que el buffer del teclado está vacío antes de ejecutar un **gets** o un **getline**. Para ello podemos usar estas funciones:

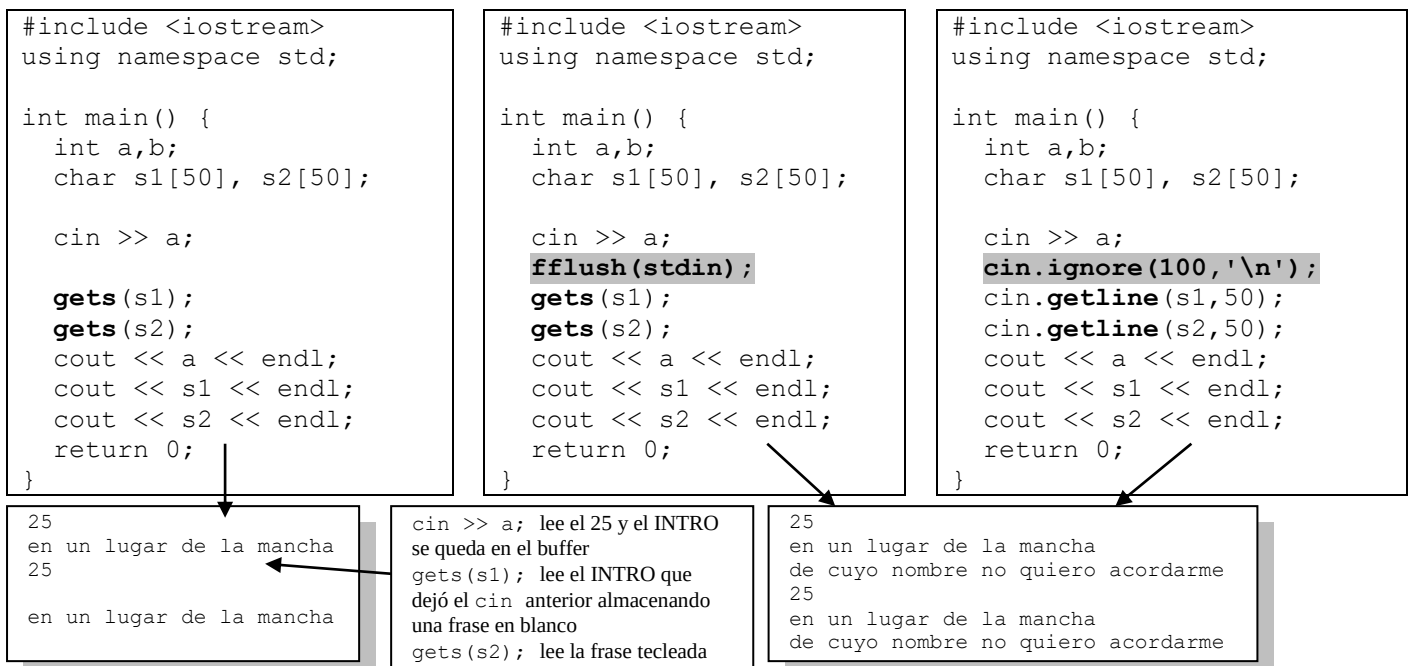
-fflush(stdin): vacia el buffer de entrada estándar (teclado)

-cin.ignore(int n = 1, int delim = EOF): ignora **n** caracteres (1 por defecto) del buffer del teclado (**cin**) o hasta encontrar el carácter indicado en **delim** (salto de línea por defecto).

```
cin.ignore(); //cin.ignore(1, '\n');
```

fflush y **gets** son de C, mientras que **ignore** y **getline** son de C++, por lo que se suelen usar en pareja.

Ej: programa con comportamiento erróneo y ese mismo programa con comportamiento correcto.



La función **fflush(stdin)** vacía por completo el buffer del teclado eliminando todo lo que hubiera allí (espacios en blanco, signos y cualquier carácter que quedara en el buffer).

El método **cin.ignore(int n = 1, int delim = EOF)** elimina los primeros **n** caracteres que haya en el buffer del teclado o hasta encontrar el caracteres delimitador **delim**.

Si sólo queremos quitar del buffer del teclado los espacios en blanco debemos usar la instrucción **cin>>std::ws**.

El manipulador **std::ws** se aplica sobre cualquier streams de entrada **istream** (**cin**, **ifstream**, etc.).

```
cin >> std::ws; //elimina los espacios en blanco que haya en el buffer de teclado
ifstream f("lista.txt"); f >> std::ws; //elimina los espacios en blanco que haya
//al inicio del fichero lista.txt
```

Si el contenido del fichero lista.txt es

```
5   Jose Luis
    Jose Maria
    Juan Antonio
    Ana
    Maria Jose
```

Supongamos que tenemos que leer un fichero lista.txt cuya primera línea es un entero que indica cuantos nombres hay en él. Nos piden hacer un programa que lea el fichero y muestre por pantalla todos los nombres, indicando al final si está todo correcto o falta algún nombre en el fichero.

```
#include <iostream>
#include <cstring> //strlen
#include <fstream> //ifstream
```

```
using namespace std;
```

```
int main() {
    int n,i;
    char s1[500];
    ifstream f("lista.txt");

    if (f.fail()) {
        cout << "Error\n";
        exit(EXIT_FAILURE);
    }
    f >> n;
    cout << "Los " << n << " nombres son:" << endl;
    i=0;
    f.ignore(100,'\n');
    while (f.getline(s1,500)) {
        cout << s1 << endl;
        i++;
    }
    f.close();
    if (i==n)
        cout << "OK: hay " << n << " nombres\n";
    else {
        cout << "ERROR: Hay " << i << " nombres\n";
        cout << "Deberia haber " << n << endl;
    }

    return 0;
};
```

```
Los 5 nombres son:
Jose Maria
Juan Antonio
Ana
Maria Jose
ERROR: Hay 4 nombres
Deberia haber 5
```

La primera línea es: 5 Jose Luis
f >> n; lee el 5 y deja en el buffer Jose Luis
f.ignore(100,'\n'); elimina Jose Luis
 el primer **f.getline(s1,500)** lee Jose Maria
 el segundo **f.getline(s1,500)** lee Juan Antonio

```
#include <iostream>
#include <cstring> //strlen
#include <fstream> //ifstream
```

```
using namespace std;
```

```
int main() {
    int ws, i;
    char s1[500];
    ifstream f("lista.txt");

    if (f.fail()) {
        cout << "Error\n";
        exit(EXIT_FAILURE);
    }
    f >> ws;
    cout << "Los " << ws << " nombres son:" << endl;
    i=0;
    f >> std::ws;
    while (f.getline(s1,500)) {
        cout << s1 << endl;
        i++;
    }
    f.close();
    if (i==ws)
        cout << "OK: hay " << ws << " nombres\n";
    else {
        cout << "ERROR: Hay " << i << " nombres\n";
        cout << "Deberia haber " << ws << endl;
    }

    return 0;
};
```

debemos usar **std::ws** por si hay por casualidad una variable llamada ws

```
f >> ws; //lee un entero
//del fichero f
//y lo guarda en ws
```

```
f >> std::ws;
//elimina espacios en blanco
//del fichero f
```

```
Los 5 nombres son:
Jose Luis
Jose Maria
Juan Antonio
Ana
Maria Jose
OK: hay 5 nombres
```

La primera línea es: 5 Jose Luis
f >> ws; lee el 5 y deja en el buffer Jose Luis
f >> std::ws; elimina que hay antes de Jose Luis
 el primer **f.getline(s1,500)** lee Jose Luis
 el segundo **f.getline(s1,500)** lee Jose Maria

Se recomienda utilizar **ignore** (o **std::ws**) y **getline** ya que además de permitir indicar el carácter delimitador (en **gets** no es posible) también permite ser usado con cualquier flujo de entrada (**istream** &) de manera que aparte del teclado (**cin**) podemos por ejemplo indicarle que lea de un fichero (**fstream**) o de un flujo de string (**stringstream**).

Contenido del fichero quijote.txt

1605

En un lugar de la mancha
de cuyo nombre no quiero acordarme
no ha mucho tiempo que vivia un hidalgo
de los de lanza en astillero,
adarga antigua, rocin flaco y galgo corredor

Ej: Programa que lee un fichero y:

- muestra cada palabra en una línea
- muestra cada frase en una línea
- muestra cada frase en una línea y enumera cada palabra de la frase (usar **blanco** como separador).

a) muestra cada palabra en una línea

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    char s1[500];
    ifstream f("quijote.txt");

    if (f.fail()) {
        cout << "Error\n";
        exit(EXIT_FAILURE);
    }

    while (f >> s1) {
        cout << s1 << endl;
    }

    f.close();
    cout << "FIN\n";

    return 0;
}
```

1605
En
un
lugar
de
la
mancha
de
cuyo
nombre
no
quiero
acordarme
...
hidalgo
de
los
de
lanza
en
astillero,
adarga
antigua,
rocin
flaco
y
galgo
corredor
FIN

b) muestra cada frase en una línea

```
#include <iostream>
#include <fstream>

using namespace std;

int main() {
    char s1[500];
    ifstream f("quijote.txt");

    if (f.fail()) {
        cout << "Error\n";
        exit(EXIT_FAILURE);
    }

    while (f.getline(s1,500)) {
        cout << s1 << endl;
    }

    f.close();
    cout << "FIN\n";

    return 0;
}
```

1605
En un lugar de la mancha
de cuyo nombre no quiero acordarme
no ha mucho tiempo que vivia un hidalgo
de los de lanza en astillero,
adarga antigua, rocin flaco y galgo
corredor
FIN

c) muestra cada frase en una línea y enumera cada palabra de la frase.

```
#include <iostream>
#include <fstream> //ifstream
#include <sstream> //stringstream
using namespace std;

int main() {
    int n;
    char s1[500], s2[500];
    ifstream f("quijote.txt");

    if (f.fail()) {
        cout << "Error al abrir fichero\n";
        exit(EXIT_FAILURE);
    }

    while (f.getline(s1,500)) {
        cout << s1 << endl;
        stringstream frase(s1);
        n=1;
        while (frase.getline(s2, 500, ' ')) {
            cout << "(" << n << " " << s2 << " ";
            n++;
        }
        cout << endl;
    }

    f.close();
    cout << "FIN\n";

    return 0;
}
```

1605
(1) 1605
En un lugar de la mancha
(1) En (2) un (3) lugar (4) de (5) la (6) mancha
de cuyo nombre no quiero acordarme
(1) de (2) cuyo (3) nombre (4) no (5) quiero (6) acordarme
no ha mucho tiempo que vivia un hidalgo
(1) no (2) ha (3) mucho (4) tiempo (5) que (6) vivia (7) un (8) hidalgo
de los de lanza en astillero,
(1) de (2) los (3) de (4) lanza (5) en (6) astillero,
adarga antigua, rocin flaco y galgo corredor
(1) adarga (2) antigua, (3) rocin (4) flaco (5) y (6) galgo (7) corredor
FIN

8.2 clase privada no instanciable (estática o singleton)

Tenemos una clase que no se debe instanciar. Sirve para agrupar semánticamente métodos y agrupar variables globales relacionadas.

Los atributos y los métodos serán static.

Evitamos la instanciación declarando el constructor por defecto como privado.

¿Qué pasa con el destructor?

Mates
+ static float global = 0
+ static const float pi = 3.14
+ static float seno()
+ void muestraNombre()

Ojo: static se pone en el .hpp pero no en el .cpp

Las variables static no se inicializan en el constructor sino fuera de la clase, al inicio del .cpp:

```
float Mates::global = 0;
const float Mates::pi = 3.14;
```

```
#include <iostream>
#include <cmath>
using namespace std;

class Mates{
private:
    Mates() { cout << "Constructor"; } //constructor privado
public:
    static float global;
    static const float pi;
    static float seno( float x ){
        return sin(x);
    }
};

float Mates::global = 0;
const float Mates::pi = 3.14;

int main(int argc, char *argv[]) {
    Mates::global = 10;
    cout << Mates::global << " : "
        << Mates::seno( Mates::pi/2 )<<"\n";
    system("PAUSE"); return EXIT_SUCCESS;
}
```

No es lo mismo clase abstracta que clase privada estática no instanciable.

Una clase que derive de una clase abstracta si se puede instanciar, pero una clase privada que derive de una clase estática no instanciable no se puede instanciar (al tener el constructor en la parte privada, la clase derivada no puede invocar dicho constructor y por tanto no puede instanciarse).

En realidad, **una clase privada puede ser instanciada mediante una función no miembro amiga de la clase**, aunque quizás no tenga mucho sentido el hacerlo:

```
#include <iostream>
#include <cmath>

using namespace std;

class Mates{
    Mates() { cout << "Constructor\n"; } //constructor privado
    ~Mates() { cout << "Destructor\n"; } //destructor privado

public:
    static float global;
    static const float pi;

    static float seno( float x ){
        return sin(x);
    }
    friend void creacion( ); //puede acceder a la parte privada
};
```

```
float Mates::global = 0;
const float Mates::pi = 3.14;

void creacion() {
    Mates x; //invoca constructor por defecto
    x.global=5;
    cout << x.global << endl;
}

int main(int argc, char *argv[]) {
    Mates::global = 10;
    cout << Mates::global << " : "
        << Mates::seno( Mates::pi/2 )<<"\n";
    creacion();
    cout << Mates::global << " : "
        << Mates::seno( Mates::pi/2 )<<"\n";
    system("PAUSE"); return EXIT_SUCCESS;
}
```

Pantalla:

```
10 : 1
Constructor
5
Destructor
5 : 1
```

8.3 Atributos mutables en Objetos constantes

Objetos y miembros constantes

Se pueden definir **objetos constantes** del mismo modo que hacemos con las variables tipos normales. Un objeto constante no puede modificarse desde el momento de su construcción hasta su destrucción (las únicas operaciones que podemos hacer es construirlos y destruirlos).

Un objeto constante sólo puede invocar (aparte de sus constructores y destructores) métodos constantes.

Los métodos estáticos no se pueden definir const.

Atributos constantes

En una clase podemos tener atributos constantes. Un atributo constante no se puede modificar una vez inicializado. Para inicializarlos obligatoriamente hay que hacerlo mediante inicializadores.

Atributos mutables

En una clase podemos definir atributos que pueden cambiar dentro de objetos constantes. Un atributo declarado mutable puede ser modificado dentro de un método constante.

```
#include <iostream>
using namespace std;
class cla {
    int a;
    const float fijo;
    mutable int i;
public:
    cla():a(0),fijo(9.8),i(0) { }
    cla(int x, float v, int y):a(x),fijo(v),i(y) { }
    int geta() const { return a; }
    float getfijo() const { return fijo; }
    int geti() const { return i; }
    void set(int x, int y) { a=x; i=y; }
    void seti(int i) const { this->i=i; }
    cla operator++() const { i++; return *this; }
    friend ostream& operator<<(ostream &s, const cla &c);
    static const cla& maxfijo(const cla &,const cla &);
};

ostream& operator<<(ostream &s, const cla &c){
    s << c.a << ":" << c.fijo << ":" << c.i;
    return s; //s actúa a modo de cout
}

const cla& cla::maxfijo(const cla &x, const cla &y) {
    if (x.fijo > y.fijo)
        return x;
    return y;
}

int main() {
    const cla a;
    cla b(2, 0.5, 3);
    cout << a.geta() << "," << a.getfijo() << "," << a.geti() << endl;
    cout << b.geta() << "," << b.getfijo() << "," << b.geti() << endl;
    b.set(6,3); //a.set(6,3); //ERROR a es constante
    a.seti(5); b.seti(4); //aunque el objeto a es constante
    ++a; ++b; //puedo modificar su atributo i
    cout << a << endl << b << endl;
    cla::maxfijo(a,b).seti(2);
    ++cla::maxfijo(a,b);
    //cla::maxfijo(a,b).set(1,2); //ERROR a es constante
    cout << a << endl << b << endl;
    system("PAUSE");return EXIT_SUCCESS;
}
```

Lo anterior es equivalente a:

```
class cla {
    ...
public:
    cla():fijo(9.8) { a=0; i=0; }
    cla(int x, float v, int y):fijo(v) {a=x; i=y;}
    ...
    friend const cla& maxfijo(const cla &, const cla &);
};

ostream& operator<<(ostream &s, const cla &c);

ostream& operator<<(ostream &s, const cla &c) {
    s << c.geta() << ":" << c.getfijo() << ":" << c.geti();
    return s; //s actúa a modo de cout
}

const cla& maxfijo(const cla &x, const cla &y) {
    if (x.fijo > y.fijo)
        return x;
    return y;
}

int main() {
    ...
    maxfijo(a,b).seti(2);
    ++maxfijo(a,b);
    //maxfijo(a,b).set(1,2); //ERROR a constnate
    cout << a << endl << b << endl;
    system("PAUSE");return EXIT_SUCCESS;
}
```

Pantalla:

```
0,9,8,0
2,0,5,3
0:9,8:6
6:0,5:5
0:9,8:3
6:0,5:5
```